



Data Orchestration & On-demand Reporting System

June-2026

Ganapathi K

Consultant, eProcurement Project, NIC-Chennai

Contents

DOORS	3
Data Orchestration & On-demand Reporting System	3
Technical, Operational & Deployment Document	3
1. Problem Definition	3
2. Proposed Solution.....	4
2.1 DOORS Architecture.....	4
2.2 DOORS Process Flow	5
2.3 DOORS - RBAC.....	6
2.4 Technology Stack & Tool Matrix	7
2.4 DOORS – Data Security.....	8
3. Operational Guide.....	9
3.1 Staging Information	9
3.2 User Accounts	12
4. DOORS Full Test Cycle	14
4.1 Data Manager Dashboard	15
4.2 Data Manager – Menus	15
4.2.1 Approve Data Requests.	15
4.2.2 Approve SQL Templates (Queries)	16
4.2.3 SQL Templates Library (Query Library)	16
4.2.4 Add / Manage Remote Agents.....	17
4.2.5 API Client Registry Management	18
4.2.5.3 IP Whitelisting.....	21
4.2.5.4 Client to SQL Mapping.....	21
4.2.5.5 Client to Agent Mapping.....	22
4.2.5.6 Generate Client API Integration Guide.....	23
4.2.5.7 Generate API Client Java Code	24
4.2.6 User to Agent Mapping	41
4.2.7 DOORS – PKI Key Management.....	43
4.2.8 New Data Request by External User	45
4.2.9 Submit SQL Templates.....	45
4.2.10 Interactive Reports (Data Reporting) – Interactive Viewer	47

4.2.11 SQL Template Category Master	47
4.3 Data Viewer Menus	48
4.4 Developer Menu.....	49
4.5 External User Menu	50
5. DOORS – Repository.....	52
6. Production Deployment.....	53
7. Major Configurations:.....	54
7.1 Session Time Out @ UI:	54
7.2 Master API Configurations.....	54
7.3 Agent API Configurations	57
7.4 Footer Customization	58
8. Online DOORS Document	59

DOORS

Data Orchestration & On-demand Reporting System

Technical, Operational & Deployment Document

DOORS is a centralized governance platform designed to manage, validate, and execute SQL queries across distributed infrastructure and share the data interactively and vi API. It bridges the gap between raw data requirements and secure, audited execution.

1. Problem Definition

The National Informatics Centre (NIC) manages critical, core state data infrastructure across multiple distributed data centers, hosting mission-critical public administration datasets including **GePNIC procurement systems, eOffice modules, eHospital frameworks, Land Records, Transport databases** and numerous other platforms.

As a growing ecosystem of external line departments, internal analytical engines, and authorized third-party entities demand real-time data integrations, traditional approaches create an severe operational bottleneck:

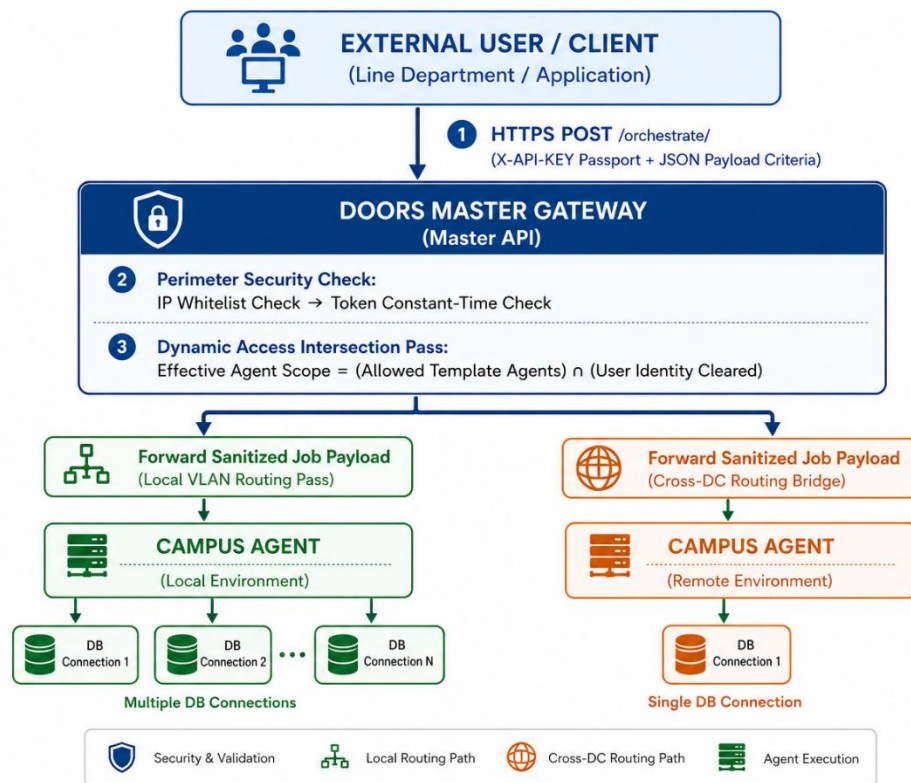
- **Direct Database Vulnerability:** Granting direct query or database connection access to external systems introduces an unacceptable security risk, exposing internal network topologies and bypassing perimeter security boundaries.
- **Monolithic Development Overhead:** Building custom, standalone REST API endpoints for every unique data request leads to unsustainably long development lifecycles, slow delivery, and rigid coupling between schemas.
- **Decentralized Management Burden:** Manually deploying, updating, and monitoring isolated integration endpoints across hundreds of fragmented regional servers creates massive configuration drift and demands a large, dedicated engineering team to maintain basic platform availability.

To bridge this operational gap, there is an urgent structural need for a centralized, zero-trust data orchestration platform. The system must enable **secure, on-demand data sharing and interactive reporting** through a unified, user-friendly management console that enforces strict security posture controls while removing manual administrative overhead.

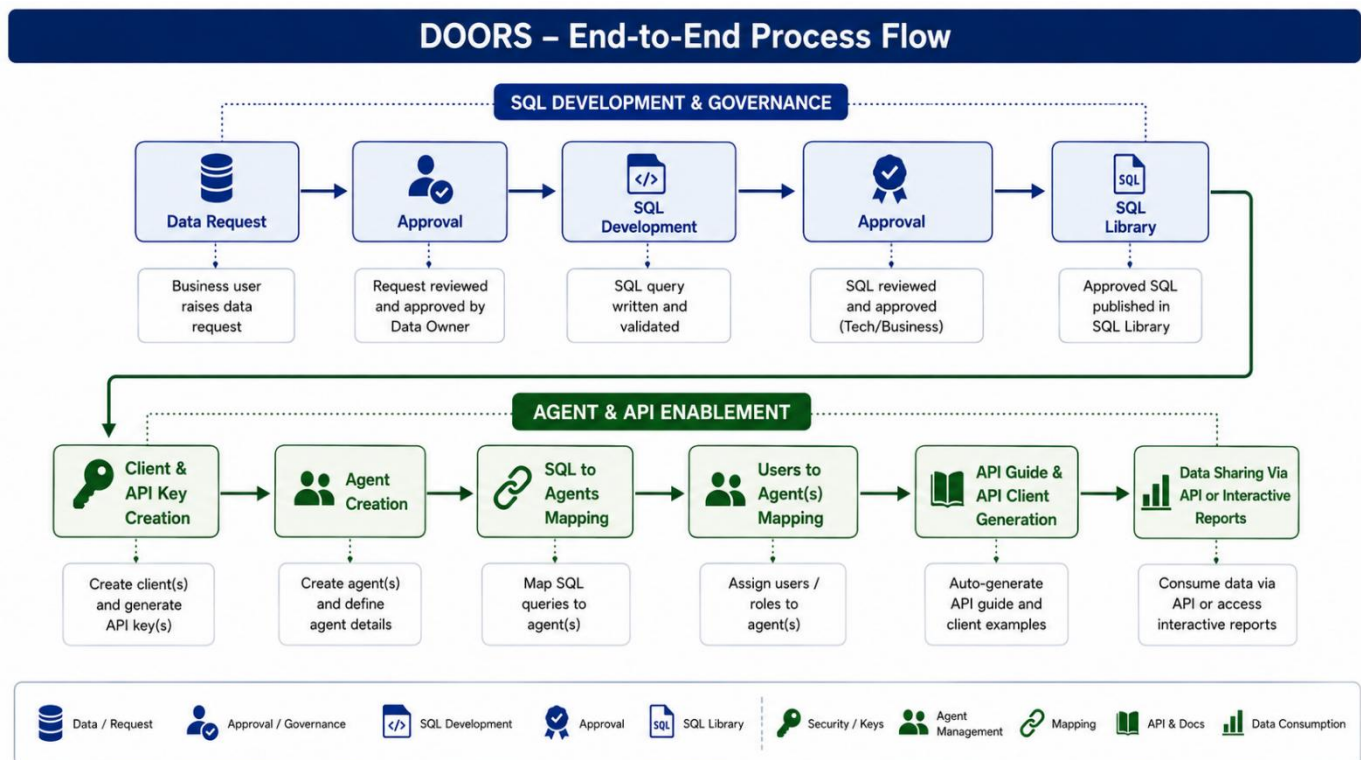
2. Proposed Solution

The proposed **Data Orchestration and On-demand Report System (DOORS)** addresses these challenges by introducing a secure, zero-trust data-bridge architecture. Rather than allowing direct database connections or by individual REST APIs, DOORS decouples data owners from data consumers through an asynchronous, agent-led pipeline.

2.1 DOORS Architecture



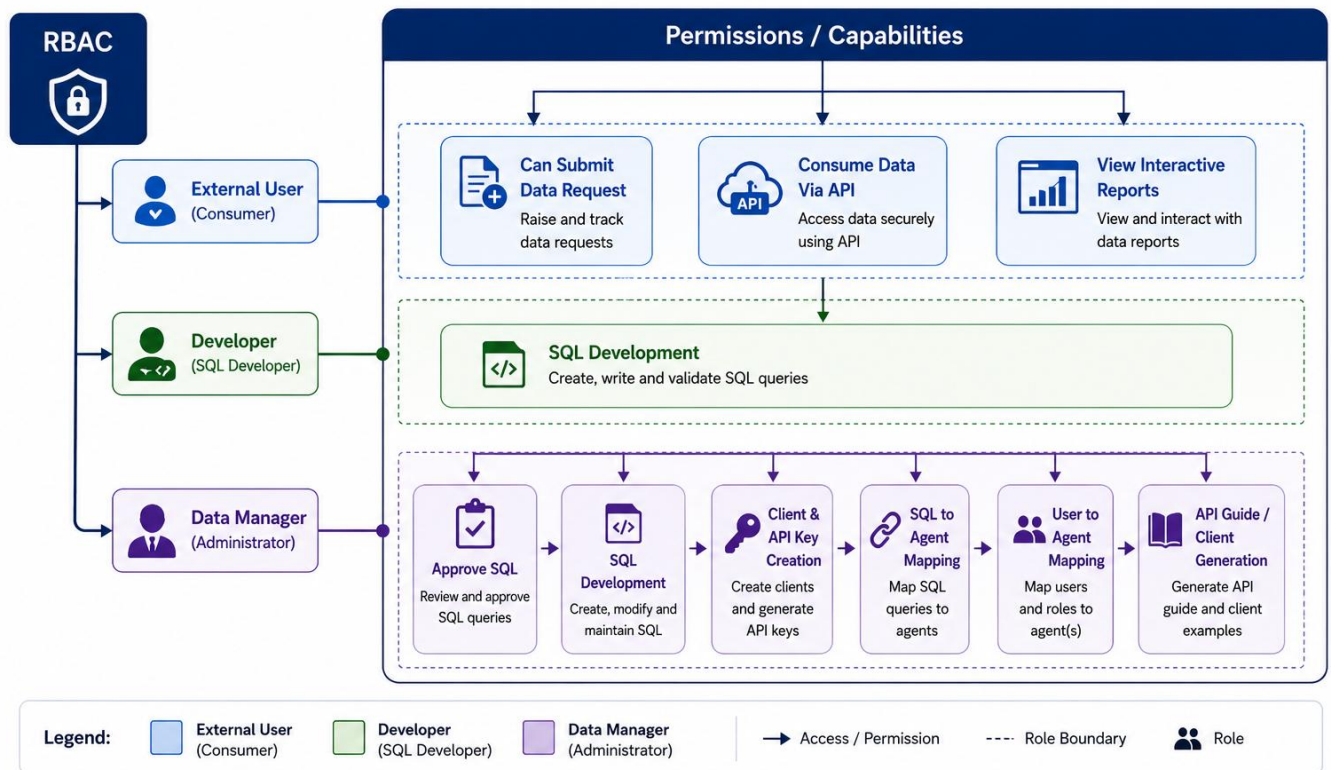
2.2 DOORS Process Flow



2.3 DOORS - RBAC

DOORS – Role Based Access Control (RBAC)

Role-based permissions for end-to-end data access, development and sharing



Role Definitions

- External User / System Client:** Can Submit data Request and view data/consume data once approved.
- Developer:** Responsible for creating and optimizing the platform's execution capability. They build raw SQL templates and assign the required parameters.
- Data Manager:** The primary compliance authority. They approve SQL templates, provision API client Key, map authorized templates to clients, and User to Agent Mapping, Client Creation, Client Public Key Mapping, API Guide & API Client Generation and Sharing.
- Data Viewer:** View Interactive Report.

2.4 Technology Stack & Tool Matrix

The entire DOORS ecosystem is built on enterprise-grade, open-source technologies to ensure compliance with open standards and long-term maintainability for the NIC teams.

Frontend Dashboard (Management Console)

- **Framework Engine:** Vue.js 3 (TypeScript Single Page Application layout)
- **UI Component Library:** Element Plus (el-table, el-dialog, el-switch styling configurations)
- **Network Client:** Axios Instance managed with custom request interception tokens
- **Export Utilities:** html2pdf.js for clean on-demand compilation of signed system integration manuals

Central Integration Engine (Master API / Gateway)

- **Core Runtime environment:** Java 21 (Long-Term Support)
- **Framework Platform Framework:** Spring Boot 3.x
- **Security Subsystem Framework:** Spring Security 6 (Stateless JWT and Custom Gateway Key Filter Engines)
- **Persistence Tier Management:** Spring Data JPA / Hibernate Core
- **High-Speed Execution Tier Data Mapper:** Spring JdbcTemplate for explicit native, type-safe SQL operations

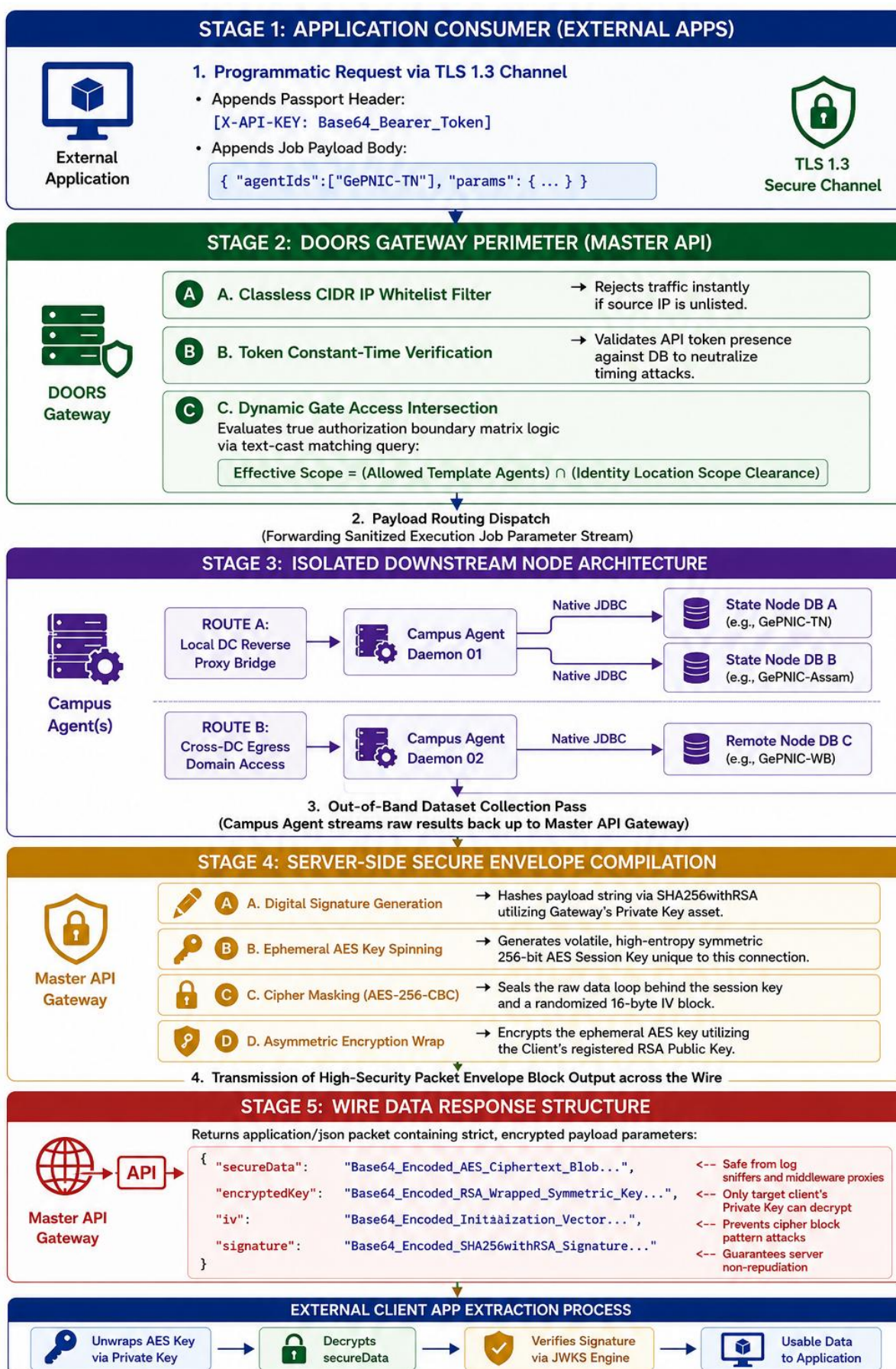
Data Layer

- **Database Engine Module:** PostgreSQL 15+

Downstream Agent Nodes (Campus Agents)

- **Web Server Layer (Front-End Reverse Proxy):** Apache httpd or Nginx
 - **Purpose:** Handles incoming corporate TLS termination, boundary network access control lists, and passes local execution proxy loops smoothly back down to the agent core.
- **Core Runtime Engine Platform (Back-End Core):** Java Spring Boot 3.x
 - **Purpose:** Maintains real-time secure outbound polling loops, executes authorized template scripts natively via decoupled driver classes, and runs isolated local query pipelines inside state data centers.

2.4 DOORS – Data Security



3. Operational Guide

3.1 Staging Information

Staging version of DOORS has been deployed in the following Infrastructure;

- UI – URL: <https://demoetenders.tn.nic.in/doors>

Build deployed under 10.246.129.226:/home/www/doors/

Backend Services are accessed by front end via Proxy settings done in /etc/httpd/gepnic/conf/modjk.conf
- Backend Master API ;
IP - 10.246.129.229
SpringBoot Jar - /home/doors/doors-master-api/doorsmasterapi.jar
Service - /etc/system/system/doorsmasterapi.service
Configuration - /home/doors/doors-master-api/config/application.yml
(To set Server Port, DB details, Connection Timeout, Idle Timeout, rate limit parameters, Strict-Origin Sites, log file)
- Database;
IP – 10.246.129.229
Port: 5455
DB : doors_registry
User: doors (DBA-gepdba)
- Staging Agent API;
IP – 10.246.129.229
Jar - /home/doors/doors-agent-api/doorsagentapi.jar
Service - /etc/systed/system/doorsagentapi.service
Configuration - /home/doors/doors-agent-api/config/application.yml
(To set White list Master API IP, log file)
- Proxy Settings for accessing the agent api via main domain;

Entry for DOORS on 22-03-2026
Timeout 1800
ProxyTimeout 1800
<Proxy "*">
Order allow,deny
Allow from all
</Proxy>

```

ProxyPreserveHost On
ProxyStatus On
ProxyTimeout 600
# Increase buffers for large data streams
ProxyIOBufferSize 65536
ProxyReceiveBufferSize 0
RequestHeader set X-Forwarded-Proto "https"
RequestHeader set X-Forwarded-Port "443"

# Force Apache to stay out of the way of the data stream
SetEnv proxy-initial-not-pooling 1
SetEnv proxy-nokeepalive 1
SetEnv force-proxy-request-1.0 1
<Location "/doorsagent">
    # Notice the path '/doorsagent' is removed from the middle of the lines
    ProxyPass http://10.246.129.229:8051/doorsagent connectiontimeout=60
    timeout=1800 disablereuse=On
    ProxyPassReverse http://10.246.129.229:8051/doorsagent
</Location>
</VirtualHost>

```

- Live Agent API;

To test with live data, doors agent api has been installed on two live GePNIC instances viz., TN & Assam;

IP – 10.246.129.97:8051 & 10.246.129.67:8051

Jar - /home/doors/doors-agent-api/doorsagentapi.jar

Service - /etc/systed/system/doorsagentapi.service

Configuration - /home/doors/doors-agent-api/config/application.yml

Proxy Settings at both the web servers of GePNIC instances are to be done so that the doors agent can be accessed via GePNIC domain like

<https://tntenders.gov.in/doorsagent>, <https://assamtenders.gov.in/doorsagent>

- **Edit /etc/httpd/gepnic/conf/modjk.conf and add the below entries Under https virtual host entries(Change the Agent API IP for every instance)**

Entry for DOORS on 22-03-2026

Timeout 1800

ProxyTimeout 1800

<Proxy "*">

Order allow,deny

Allow from all

</Proxy>

ProxyPreserveHost On

ProxyStatus On

ProxyTimeout 600

Increase buffers for large data streams

ProxyIOBufferSize 65536

ProxyReceiveBufferSize 0

RequestHeader set X-Forwarded-Proto "https"

RequestHeader set X-Forwarded-Port "443"

Force Apache to stay out of the way of the data stream

SetEnv proxy-initial-not-pooling 1

SetEnv proxy-nokeepalive 1

SetEnv force-proxy-request-1.0 1

<Location "/doorsagent">

Notice the path '/doorsagent' is removed from the middle of the lines

ProxyPass http://10.246.129.97:8051/doorsagent connectiontimeout=60
timeout=1800 disablereuse=On

ProxyPassReverse http://10.246.129.97:8051/doorsagent

</Location>

</VirtualHost>

3.2 User Accounts

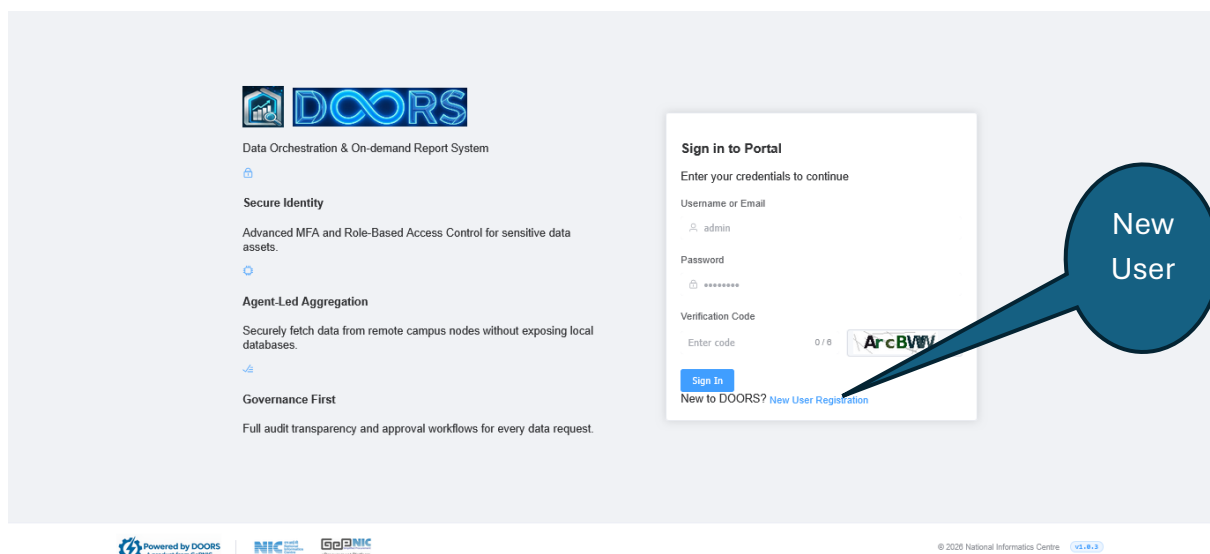
The staging version of DOORS by default comes with the following users created for completing the testing before the production deployment – default password doors@nic

- datamanager@gmail.com
- dataviewer@gmail.com
- externaluser@gmail.com
- developer@gmail.com

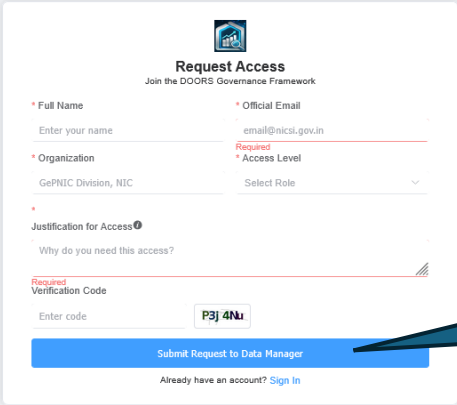
On first login, the user will be forced to set new password.

3.2.1 Adding a new user:

New user must submit the request through ‘[New Registration Request](#)’ on the DOORS home page.



Back to Home



Request Access
Join the DOORS Governance Framework

* Full Name
Enter your name

* Official Email
email@nicl.gov.in

* Organization
GePNIC Division, NIC

* Access Level
Required

* Select Role

Justification for Access
Why do you need this access?

Required Verification Code
Enter code

Submit Request to Data Manager

Already have an account? [Sign In](#)

Submit to Data Manager

Data Manager's User Approval Form

Login as 'Data Manager' -> User Management -> Pending Approval -> Click on 'Approve'

DOORS Data Manager

My Dashboard
New Data Request
My Data Requests
SQL Submission
Governance Hub
Approve Data Requests
Approve Queries
Query Library
Remote Agents
API Client Registry
Category Master
User Management
User-Agent Matrix
Execution Logs

demoetenders.tn.nic.in/doors/dashboard/governance/users

80% Sign in

DOORS Data Orchestrator and On-demand Reporting System

GOVERNANCE INTEGRATION DATA SHARING

Data Manager MODE

datamanager@gmail.com

Identity & Access Governance

Refresh Directory

Pending Requests: 1 Active Personnel: 7 Rejected Audit: 0

Approvals Needs Active Directory Rejected History

USERNAME	FULL NAME	DEPARTMENT	REQ. ROLE	GOVERNANCE
test@nic.in	Test	GePNIC	External	Approve Reject

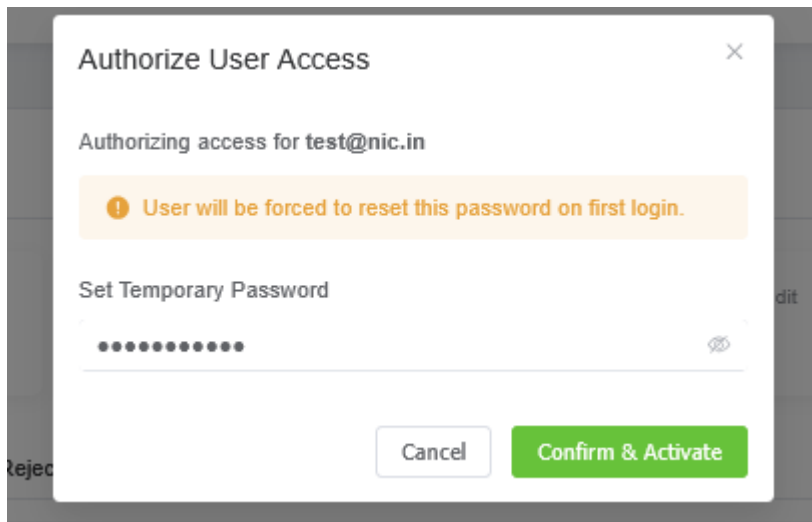
Approve

Powered by DOORS A product from GePNIC

NIC GePNIC

Support Terms of Service Privacy Policy

© 2026 National Informatics Centre v1.0.1



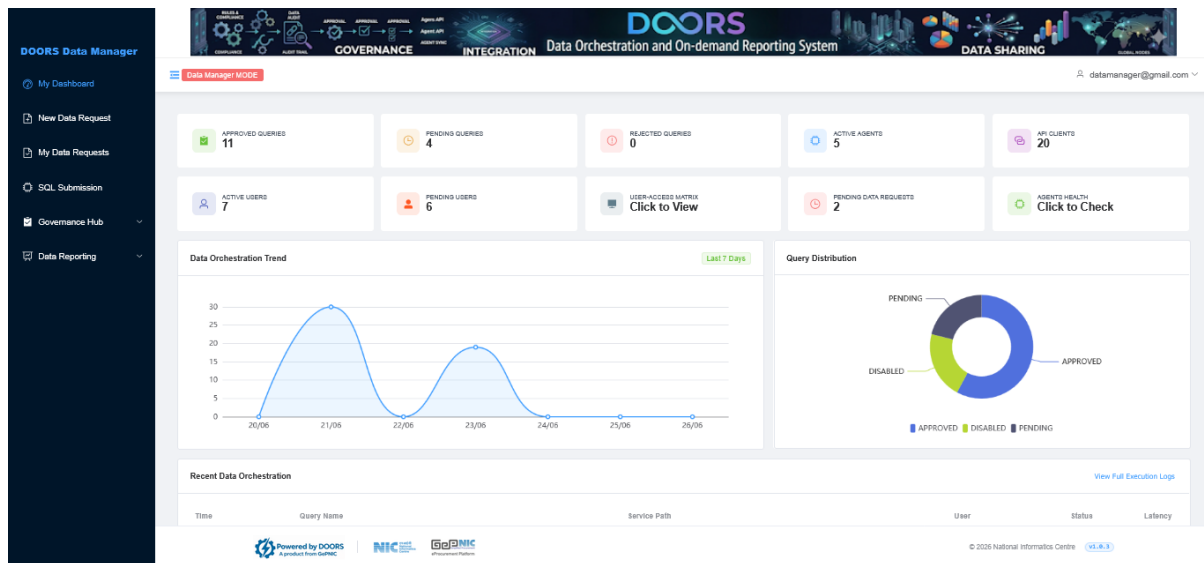
The new user will be forced to change the password on first login.

4. DOORS Full Test Cycle

Steps to carry out the functional test of DOORS;

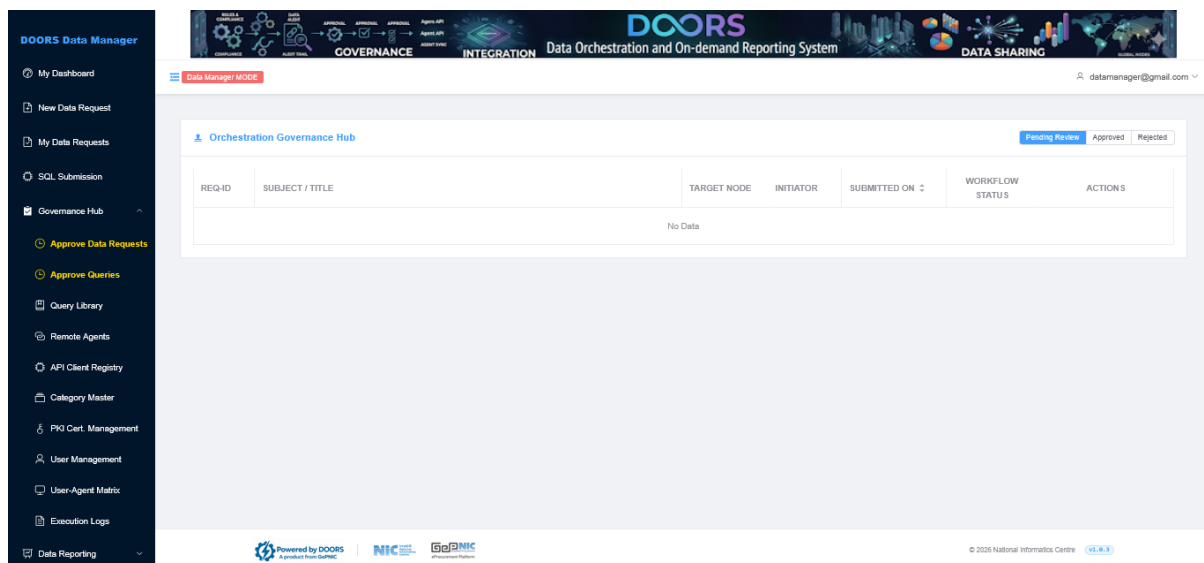
- i. Login as 'externaluser@gmail.com' and submit the data request.
- ii. Login as 'datamanager@gmail.com' and approve the data request.
- iii. Login as 'developer@gmail.com' and develop the SQL and submit for approval.
- iv. Login as 'datamanager@gmail.com' and approve the SQL, Map the SQL to a GePNIC Instance (Agent), Map the user 'dataviewer@gmail.com' to the same GePNIC Instance.
- v. Login as 'dataviewer@gmail.com' and Click 'Data Reporting'-'>'Interactive Viewer'
- vi. Again, login as 'datamanager@gmail.com' -> Api Client Registry-> Register New Client . After adding API Client, Add Client Public Key, Map SQL & GePNIC Instance(s) to that API.
Create 'Guide'. Copy the curl URL from the Guide and test it via api client POSTMAN.
Create 'API Client' which can be shared to the end user who can customize it and use it instantly for fetching the data.

4.1 Data Manager Dashboard



4.2 Data Manager – Menus

4.2.1 Approve Data Requests.



4.2.2 Approve SQL Templates (Queries)

DOORS Data Manager

Pending SQL Approvals

Query Name	Category	Description	Requested By	Governance
SMS-Count-Sept-2021	INFRA	To Test Random Query	admin	Review & Approve
Daily_Tender_Summary	MIS	Dry Run Tested.	admin	Review & Approve
Test1	MIS	Tested	admin	Review & Approve
UPPTCL	ON_DEMAND	Tested	admin	Review & Approve

© 2020 National Informatics Centre

4.2.3 SQL Templates Library (Query Library)

DOORS Data Manager

DOORS Query Governance Library

QUERY NAME	CATEGORY	PROPO SER	AUTHORIZED NODES	SQL	STATUS	ACTIONS
AOC_DATA_WITHOUT_JSON_AGG	ON_DEMAND	datamanager@gmail.com	Dev-01	View SQL	APPROVED	Mappings
ToTestCentralAPI	MIS	datamanager@gmail.com	Dev-01 GePNIC-Staging-2 GePNIC-Assam	View SQL	APPROVED	Mappings
AOC_BY_TENDERID	ON_DEMAND	developer@g mail.com	Dev-01	View SQL	APPROVED	Mappings
AOC-JSON	ON_DEMAND	admin	Dev-01	View SQL	APPROVED	Mappings
Query_Submitted_By_External_User	ON_DEMAND	ganapathik@gmail.com	Dev-01	View SQL	APPROVED	Mappings
ORGWISE_USERS	MIS	admin	Dev-01 Dev-02 GePNIC-Assam GePNIC-Staging-2	View SQL	APPROVED	Mappings
AOC_DATA	ON_DEMAND	developer@g mail.com	Dev-01 GePNIC-Staging-2 GePNIC-Assam	View SQL	APPROVED	Mappings
Bidders List	MIS	admin	Dev-01 Dev-02	View SQL	APPROVED	Mappings
DAILY_TENDER_SUMMARY	MIS	admin	Dev-01	View SQL	APPROVED	Mappings

© 2020 National Informatics Centre

4.2.4 Add / Manage Remote Agents

- Add remote Agent APIs
- Model Type Central → Single API can be mapped to multiple DBs with same domain and with different end points.

(<https://demoetenders.tn.nic.in/doorsagent/staging-2>
<https://demoetenders.tn.nic.in/doorsagent/assam>)

- **Model Type Individual** -> **Single API mapped to single DB via application.yml file (For Instances running of Different DC from Master API DC)**

The screenshot displays the DOORS Data Manager interface. The left sidebar contains navigation links: My Dashboard, New Data Request, My Data Requests, SQL Submission, Governance Hub, Approve Data Requests, Approve Queries, Query Library, Remote Agents, API Client Registry, Category Master, PKI Cert. Management, User Management, User-Agent Matrix, Execution Logs, and Data Reporting. The main content area shows the 'Instance Master: GePNIC Campus Registry' table. The table has columns for Agent ID, Agent Display Name, Model Type, DB Host, DB Port, DB Name, DB User, DB Password, Agent Status URL, Environment, Status, and Actions. The table lists five agents: Dev-02, Dev-01, GePNIC-Staging-2, GePNIC-Assam, and GePNIC-UAT. The GePNIC-UAT agent is marked as 'INDIVIDUAL' and 'OFFLINE'.

Agent ID	Agent Display Name	Model Type	DB Host	DB Port	DB Name	DB User	DB Password	Agent Status URL	Environment	Status	Actions
Dev-02	Dev-02	CENTRAL	localhost	5433	demoeproc	gepuser	-	http://127.0.0.1:8051/dev-02	PROD	ONLINE	[Icons]
Dev-01	Development Environment - 01	CENTRAL	localhost	5433	demoeproc	gepuser	-	http://127.0.0.1:8051/dev-01	SANDBOX	ONLINE	[Icons]
GePNIC-Staging-2	GePNIC Staging 2	CENTRAL	10.248.129.227	5433	gepniv_v23_m p	gepuser	-	https://demoetenders.tn.nic.in/doorsagent/staging-2	PROD	ONLINE	[Icons]
GePNIC-Assam	GePNIC-Assam	CENTRAL	10.248.129.88	5433	gepasam	misuser	-	https://demoetenders.tn.nic.in/doorsagent/assam	PROD	ONLINE	[Icons]
GePNIC-UAT	UAT	INDIVIDUAL	-	-	-	-	-	http://demoeproc.nic.in	PROD	OFFLINE	[Icons]

At the bottom of the interface, there are logos for 'Powered by DOORS', 'NIC', and 'GePNIC', along with the text '© 2025 National Informatics Centre' and 'v1.9.3'.

4.2.5 API Client Registry Management

- Register New Client (API Key gets generated Automatically)
- Map Client's Public Key (Button -> 'Public Key' under Governance Column)
- Secure the Access for this client from a whitelisted IP (Button -> 'IP Whitelist')
- Do the Client to SQL Template(s) Mapping (Query Mapping) (Button -> 'Queries')
- Do the Client to Agent Mapping (Button -> 'Agents')
- Generate Integration Guide to be shared to end user (Button -> 'Guide')
- Generate the API Client Java Code to be shared to end user (Button -> 'Client API')
- Enable/Disable Encryption (Test with PLAIN option first, then enable AES Option)

GOVERNANCE

Guide

Client API

IP Whitelist

Public Key

Queries

Agents

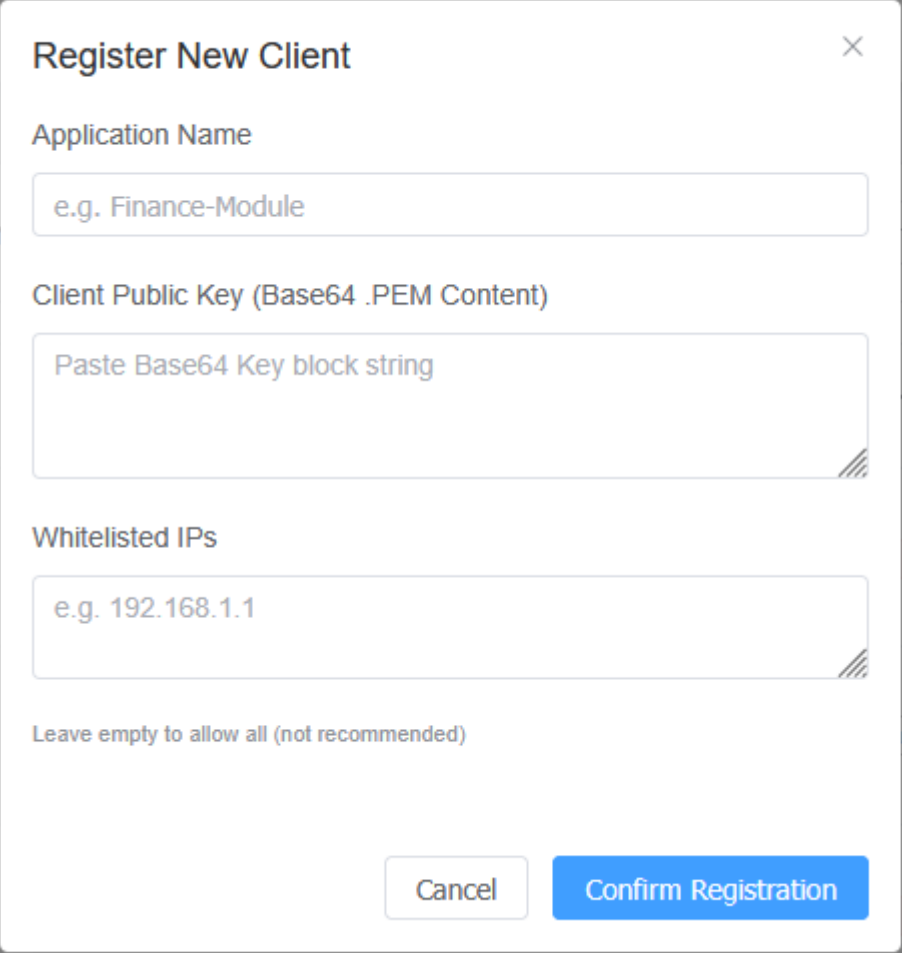
DOORS Data Manager

API Client Registry (External Gateway)

ID	APPLICATION NAME	X-API-KEY (PASSPORT)	CLIENT PUBLIC KEY	WHITELISTED IPs	AUTHORIZED QUERIES	AGENT SCOPE	ENCRYPTION	GOVERNANCE
10	admin		Missing Key	Any (Not Secure)	No Authorizations	None	PLAIN GRACE PERIOD	Guide Client API IP Whitelist Public Key Queries Agents
1	GePNIC-Internal		Configured (masked)	localhost	ORGWISE_USERS Bidders List	Dev-01	AES ACTIVE	Guide Client API IP Whitelist Public Key Queries Agents
11	GePNIC-New		Missing Key	Any (Not Secure)	Bidders List	Dev-01	PLAIN GRACE PERIOD	Guide Client API IP Whitelist Public Key Queries Agents
12	GePNIC-New2		Missing Key	Any (Not Secure)	AOC_DATA Bidders List	Dev-01 GePNIC-Assam	AES ACTIVE	Guide Client API IP Whitelist Public Key Queries Agents
13	GePNIC-New3		Missing Key	Any (Not Secure)	No Authorizations	None	AES ACTIVE	Guide Client API IP Whitelist Public Key Queries Agents
14	GePNIC-New4		Missing Key	Any (Not Secure)	No Authorizations	None	AES ACTIVE	Guide Client API IP Whitelist Public Key Queries Agents

Powered by DOORS A product from GePNIC | NIC | GePNIC
© 2025 National Informatics Centre v1.4.3

4.2.5.1 Register New Client



The image shows a 'Register New Client' dialog box with a close button (X) in the top right corner. It contains three input fields: 'Application Name' with the placeholder 'e.g. Finance-Module', 'Client Public Key (Base64 .PEM Content)' with the placeholder 'Paste Base64 Key block string', and 'Whitelisted IPs' with the placeholder 'e.g. 192.168.1.1'. Below the third field is the text 'Leave empty to allow all (not recommended)'. At the bottom are 'Cancel' and 'Confirm Registration' buttons.

Register New Client ✕

Application Name

e.g. Finance-Module

Client Public Key (Base64 .PEM Content)

Paste Base64 Key block string

Whitelisted IPs

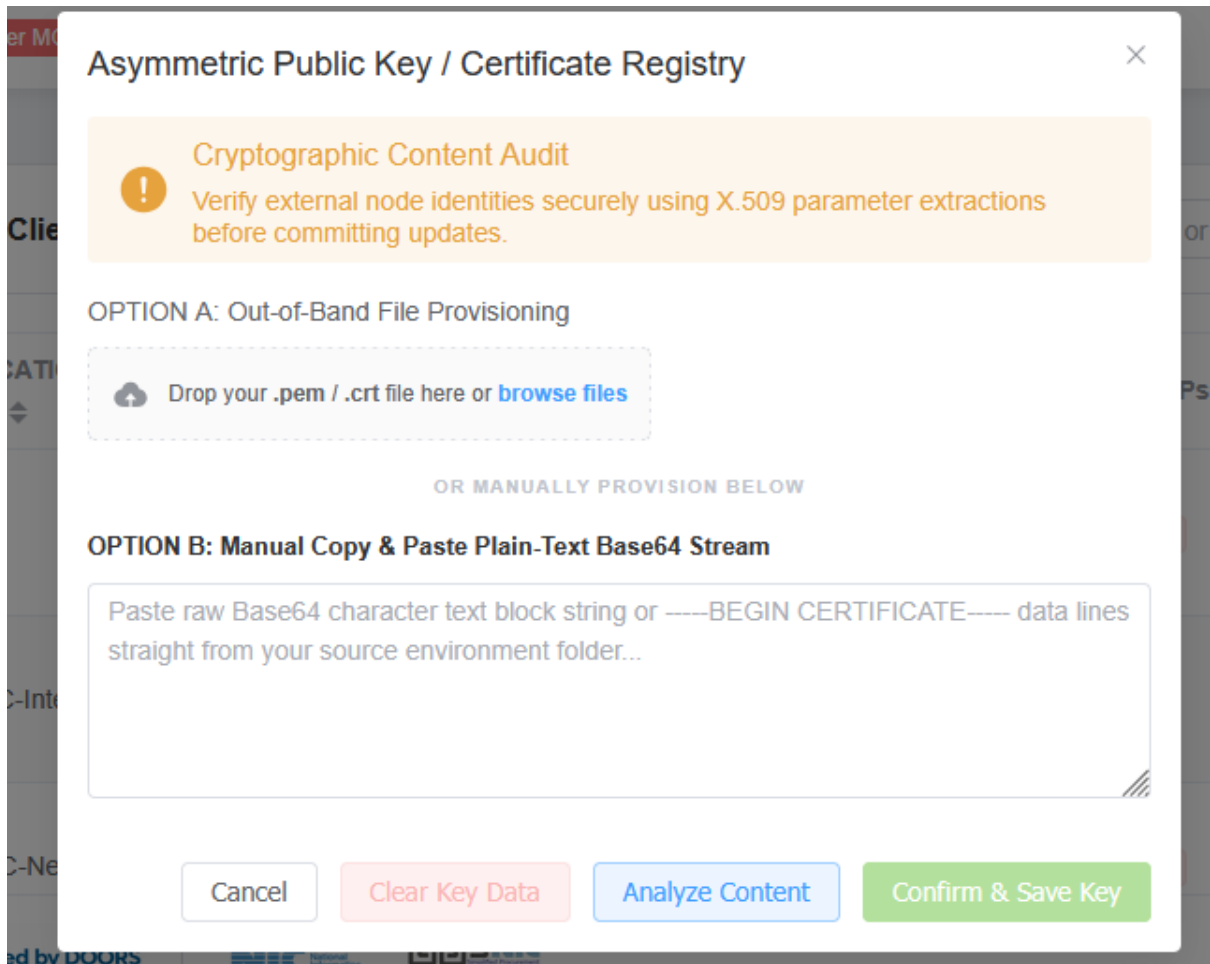
e.g. 192.168.1.1

Leave empty to allow all (not recommended)

Cancel Confirm Registration

4.2.5.2 Map Client Public Key

- Can Upload .pem file or Can paste the Certificate -> 'Analyze the Content' and then 'Confirm & Save Key'
- Can delete the mapped key by 'Clear Key Data' button



Asymmetric Public Key / Certificate Registry

Cryptographic Content Audit
Verify external node identities securely using X.509 parameter extractions before committing updates.

OPTION A: Out-of-Band File Provisioning

Drop your .pem / .crt file here or [browse files](#)

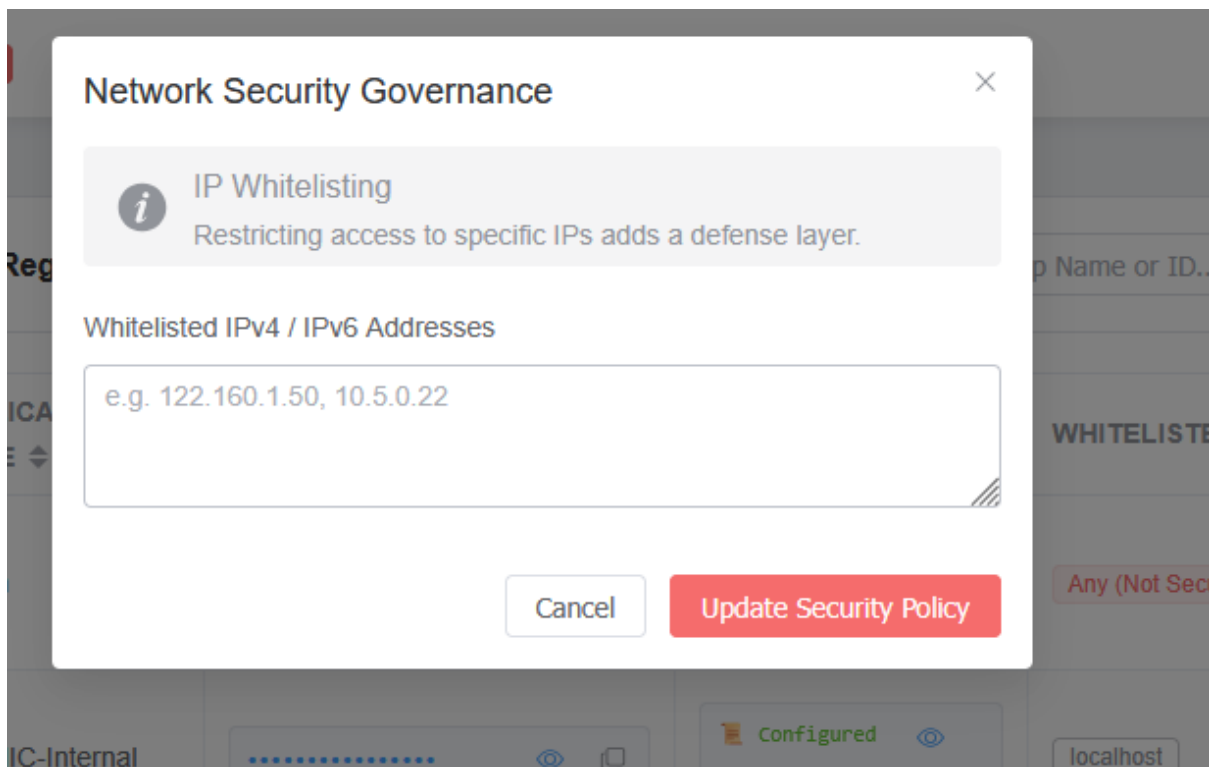
OR MANUALLY PROVISION BELOW

OPTION B: Manual Copy & Paste Plain-Text Base64 Stream

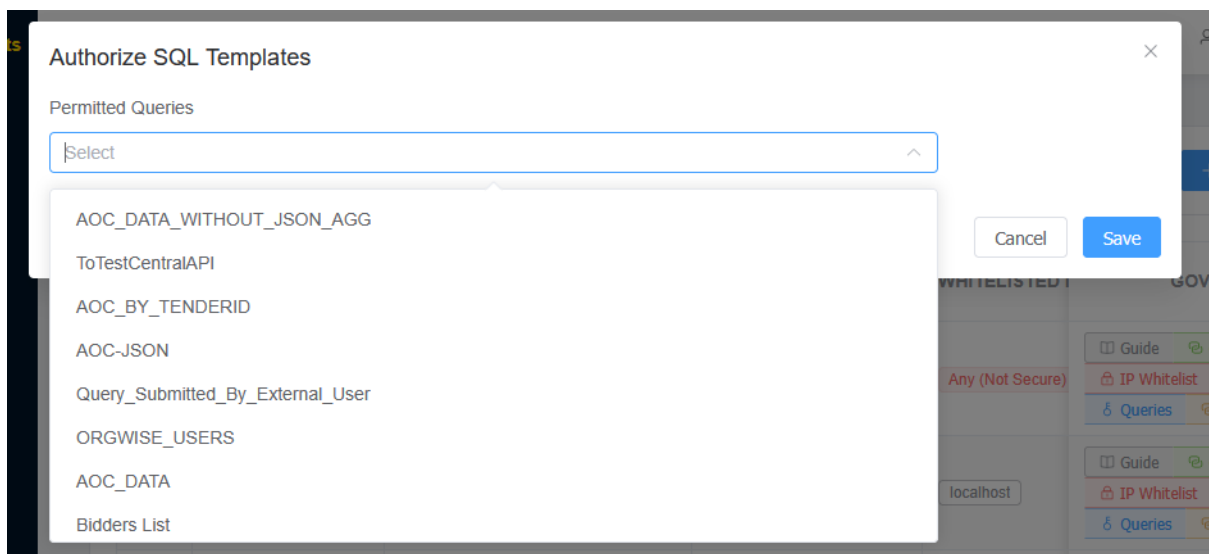
Paste raw Base64 character text block string or -----BEGIN CERTIFICATE----- data lines straight from your source environment folder...

Cancel Clear Key Data Analyze Content Confirm & Save Key

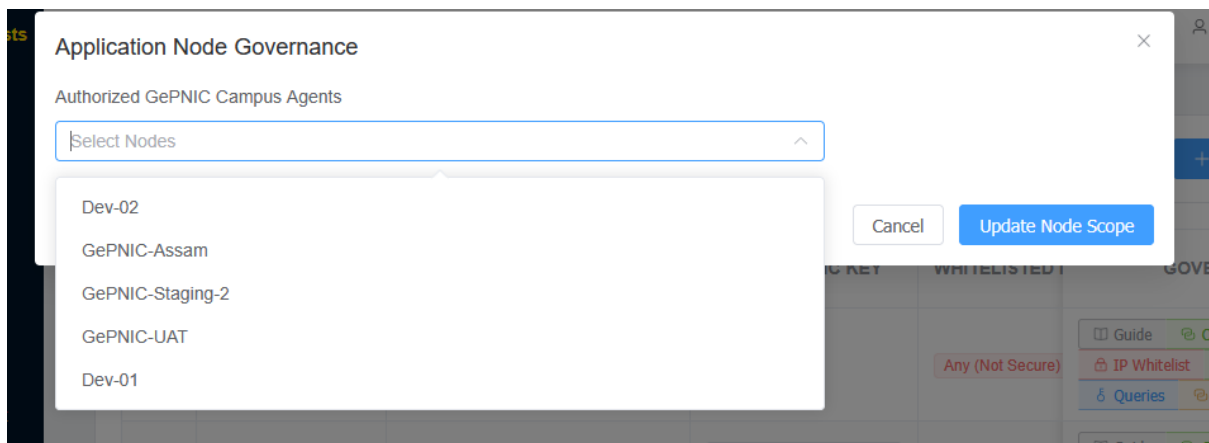
4.2.5.3 IP Whitelisting



4.2.5.4 Client to SQL Mapping



4.2.5.5 Client to Agent Mapping



4.2.5.6 Generate Client API Integration Guide

API Integration Guide

Save as PDF

X

DOORS Governance Framework

Generated: 6/26/2026, 7:45:01 PM

!

Security Mandate

Identity resolved via server-side passport mapping.

Application: GePNIC-Internal

Security Header:X-API-KEY: obxXlht2wcZF9FDAG6i4EGxlyFMAXmxZ

Data Format Strategy: AES-256 ECB Encrypted (Decryption Required)

Authorized Source IPs:

localhost

Requests from any other IP address will be rejected at DOORS.

Authorized Endpoints

ORGWISE_USERS

Copy Curl Command

```
curl -X POST "http://localhost:8052/api/v1/master/gateway/orchestrate/ORGWISE_USERS" \
-H "X-API-KEY: obxXlht2wcZF9FDAG6i4EGxlyFMAXmxZ" \
-H "Content-Type: application/json" \
-d '{
  "agentIds": [
    "Dev-01"
  ],
  "params": {}
}'
```

Bidders List

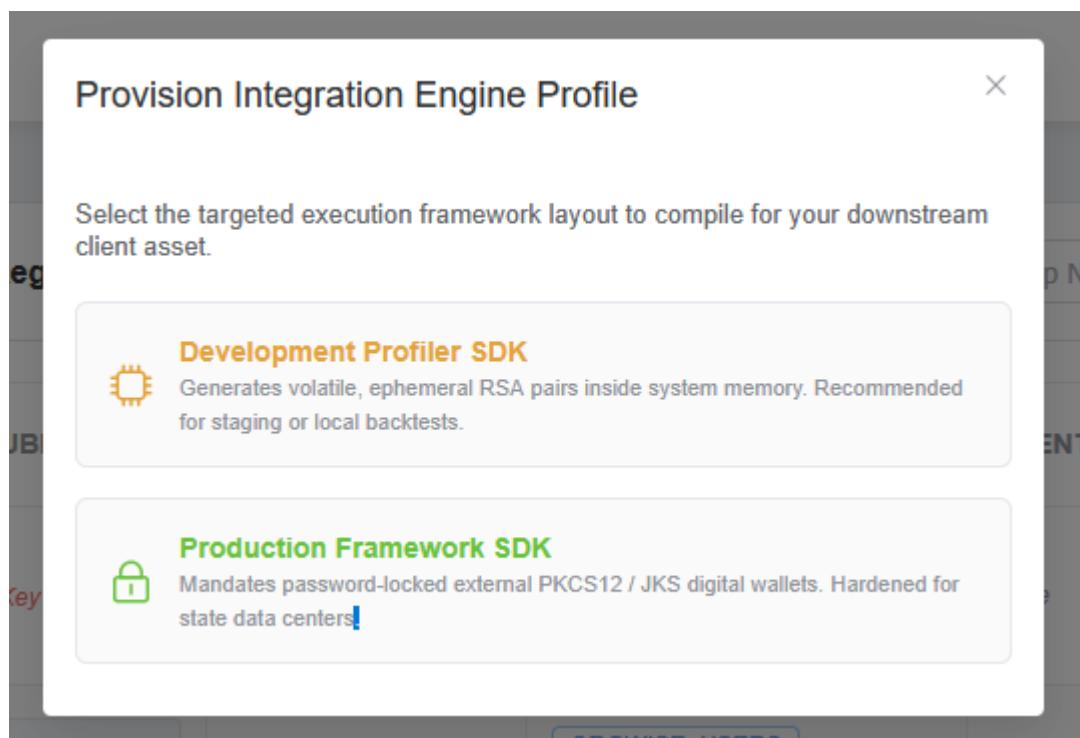
Copy Curl Command

```
curl -X POST "http://localhost:8052/api/v1/master/gateway/orchestrate/Bidders List" \
-H "X-API-KEY: obxXlht2wcZF9FDAG6i4EGxlyFMAXmxZ" \
-H "Content-Type: application/json" \
-d '{
  "agentIds": [
    "Dev-01"
  ],
  "params": {}
}'
```

4.2.5.7 Generate API Client Java Code

- Two types of Client can be generated

- (i) Development Profiler SDK – This can be shared with the user to do the testing on our UAT/Staging – This does not require PKI – Public Key and Private key at the user end.
- (ii) Production Framework SDK – This can be shared with the user for production use – Requires PKI – Public Key and Private key at user end. User must share the Public key to ‘Data Manager’ of DOORS and should be mapped against the Client.



4.2.5.7.1 Development Version of API Client

```
/**
 *
 =====
 =====
 * 🇧🇩 NATIONAL INFORMATICS CENTRE (NIC) — GePNIC - Data Orchestration API
 Client
 * 📄 STANDARD OPERATING PROCEDURE (SOP) FOR CRYPTOGRAPHIC ASSET
 MANAGEMENT
 * Disclaimer: This generated client template is provided as a reference implementation
 * to demonstrate secure integration practices with the DOORS API. It is the
 * responsibility of the implementer to ensure that all cryptographic materials
 * are managed in accordance with organizational security policies and industry
 * best practices.
 *
 =====
 =====
 * * 1. IDENTITY & CONFIGURATION SUMMARY
 * - Client Class: GePNICDOORSAPIClientDevProfileGePNICInternal
 * - Profile Mode: DEVELOPMENT PROFILE (Ephemeral In-Memory)
 * - Security Model: Digital Certificate Handshake with Outbound Data Payload
 Encryption
 * *
 * * 2. LOCAL TESTING BOUNDARY PROVISIONING (EPHEMERAL IDENTITY)
 * - Ephemeral RSA key pairs are generated automatically inside application execution
 memory.
 * - Securely capture the API key passport assigned to your node context by your API
 service provider.
 * - Provision your tracking credential locally using the following isolated terminal
 routines:
 * [Linux/Unix Terminal]:
 * $ mkdir -p ~/security/
 * $ echo "API_KEY_SHARED_BY_API_SERVICE_PROVIDER" > ~/security/doors.key
 * $ chmod 600 ~/security/doors.key
 * * [Windows PowerShell]:
 * > md "$HOME\security"
 * > "API_KEY_SHARED_BY_API_SERVICE_PROVIDER" | Out-File -FilePath
 "$HOME\security\doors.key" -Encoding ascii
 * * 3. COMPILATION & TESTING INSTRUCTIONS
 * Step 1 (Compile & Build Local Bytecode Folder Tree):
 * $ javac -d . GePNICDOORSAPIClientDevProfileGePNICInternal.java
 * * Step 2 (Execute fully qualified class payload):
 * $ java
 org.gepnic.doors.masterapi.GePNICDOORSAPIClientDevProfileGePNICInternal
```

```

* *
=====
=====
*/
package org.gepnic.doors.masterapi;

import javax.crypto.Cipher;
import javax.crypto.spec.IvParameterSpec;
import javax.crypto.spec.SecretKeySpec;
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.OutputStream;
import java.net.HttpURLConnection;
import java.net.URL;
import java.net.URLEncoder;
import java.math.BigInteger;
import java.nio.charset.StandardCharsets;
import java.security.KeyFactory;
import java.security.KeyPair;
import java.security.PublicKey;
import java.security.spec.RSAPublicKeySpec;
import java.util.Base64;
import java.util.List;

public class GePNICDOORSAPIClientDevProfileGePNICInternal {

    private final String baseUrl;
    private final String apiKey;
    private KeyPair liveKeyPair;
    private String publicKeyBase64;

    private PublicKey cachedDoorsPublicKey = null;
    private String cachedDoorsKeyId = null;

    public GePNICDOORSAPIClientDevProfileGePNICInternal(String baseUrl, String
apiKey) throws Exception {
        this.baseUrl = baseUrl.endsWith("/") ? baseUrl.substring(0, baseUrl.length() - 1) :
baseUrl;
        this.apiKey = apiKey;
        generateVolatileKeyPair();
    }

    private void generateVolatileKeyPair() throws Exception {
        java.security.KeyPairGenerator keyGen =
java.security.KeyPairGenerator.getInstance("RSA");
        keyGen.initialize(2048, new java.security.SecureRandom());
        this.liveKeyPair = keyGen.generateKeyPair();
    }

```

```

        this.publicKeyBase64 =
Base64.getEncoder().encodeToString(this.liveKeyPair.getPublic().getEncoded());
    }

    public void executeHandshake() throws Exception {
        HttpURLConnection conn = (HttpURLConnection) new URL(this.baseUrl +
"/handshake").openConnection();
        conn.setRequestMethod("POST");
        conn.setRequestProperty("Content-Type", "application/json");
        conn.setRequestProperty("X-API-KEY", this.apiKey);
        conn.setDoOutput(true);

        String payload = String.format("{\"publicKey\": \"%s\"}", this.publicKeyBase64);
        try (OutputStream os = conn.getOutputStream()) {
            os.write(payload.getBytes(StandardCharsets.UTF_8));
        }

        if (conn.getResponseCode() != 200) {
            throw new RuntimeException("DOORS-HANDSHAKE-REJECTED.");
        }
    }

    private synchronized PublicKey resolveDoorsPublicKey(String requiredKeyId) throws
Exception {
        if (this.cachedDoorsPublicKey != null &&
requiredKeyId.equals(this.cachedDoorsKeyId)) {
            return this.cachedDoorsPublicKey;
        }

        HttpURLConnection conn = (HttpURLConnection) new URL(this.baseUrl + "/.well-
known/jwks.json").openConnection();
        conn.setRequestMethod("GET");
        conn.setRequestProperty("X-API-KEY", this.apiKey);
        String jwksResponse = readStream(conn.getInputStream());

        String base64UrlModulus = extractJsonValue(jwksResponse, "n");
        byte[] modulusBytes = Base64.getUrlDecoder().decode(base64UrlModulus);
        byte[] exponentBytes =
Base64.getUrlDecoder().decode(extractJsonValue(jwksResponse, "e"));

        BigInteger modulus = new BigInteger(1, modulusBytes);
        BigInteger exponent = new BigInteger(1, exponentBytes);

        RSAPublicKeySpec rsaSpec = new RSAPublicKeySpec(modulus, exponent);
        this.cachedDoorsPublicKey =
KeyFactory.getInstance("RSA").generatePublic(rsaSpec);
        this.cachedDoorsKeyId = requiredKeyId;
    }

```

```

        return this.cachedDoorsPublicKey;
    }

    public String fetchReportData(String queryUniqueName, List<String> targetAgentIds,
String criteriaParams) throws Exception {
        String encodedQuery = URLEncoder.encode(queryUniqueName,
StandardCharsets.UTF_8.toString()).replaceAll("\\\\+", "%20");
        HttpURLConnection conn = (HttpURLConnection) new URL(this.baseUrl +
"/orchestrate/" + encodedQuery).openConnection();
        conn.setRequestMethod("POST");
        conn.setRequestProperty("Content-Type", "application/json");
        conn.setRequestProperty("X-API-KEY", this.apiKey);
        conn.setDoOutput(true);

        StringBuilder agentsJson = new StringBuilder();
        agentsJson.append("[");
        for (int i = 0; i < targetAgentIds.size(); i++) {
            agentsJson.append("\"").append(targetAgentIds.get(i).trim()).append("\"");
            if (i < targetAgentIds.size() - 1) {
                agentsJson.append(",");
            }
        }
        agentsJson.append("]");

        String cleanParams = (criteriaParams == null || criteriaParams.trim().isEmpty()) ? "{}"
: criteriaParams.trim();
        String jsonRequestPayload = "{\"agentIds\":\"" + agentsJson.toString() + "\",\"params\":\""
+ cleanParams + "\"}";

        try (OutputStream os = conn.getOutputStream()) {
            os.write(jsonRequestPayload.getBytes(StandardCharsets.UTF_8)); }

        int status = conn.getResponseCode();
        if (status == 403) {
            executeHandshake();
            return fetchReportData(queryUniqueName, targetAgentIds, criteriaParams);
        }

        if (status >= 400) {
            throw new RuntimeException("DOORS Error -> " +
readStream(conn.getErrorStream()));
        }

        String serverKeyIdHint = conn.getHeaderField("X-DOORS-KID");
        String response = readStream(conn.getInputStream());

        String secureDataBase64 = extractJsonValue(response, "secureData");

```

```

String encryptedKeyBase64 = extractJsonValue(response, "encryptedKey");
String ivBase64 = extractJsonValue(response, "iv");
String signatureBase64 = extractJsonValue(response, "signature");

if (secureDataBase64 == null) return response;

// 🚀 MODULE 1: INBOUND WIRE DATA VERIFICATION PANEL

System.out.println("\n=====
=====");
    System.out.println("🚀 INBOUND WIRE DATA VERIFICATION (RAW ENCRYPTED
BLOCKS)");

System.out.println("=====
=====");
    System.out.println("🔒 Encrypted Session AES Key : " +
truncateSignatureString(encryptedKeyBase64));
    System.out.println("🔑 Initialization Vector (IV): " + ivBase64);
    System.out.println("📦 Ciphertext Payload Body : " +
truncateSignatureString(secureDataBase64));

System.out.println("=====
=====");

    Cipher rsaCipher = Cipher.getInstance("RSA/ECB/PKCS1Padding");
    rsaCipher.init(Cipher.DECRYPT_MODE, this.liveKeyPair.getPrivate());
    byte[] rawAesKeyBytes =
rsaCipher.doFinal(Base64.getDecoder().decode(encryptedKeyBase64));

    Cipher aesCipher = Cipher.getInstance("AES/CBC/PKCS5Padding");
    SecretKeySpec secretKey = new SecretKeySpec(rawAesKeyBytes, "AES");
    IvParameterSpec ivSpec = new
IvParameterSpec(Base64.getDecoder().decode(ivBase64));
    aesCipher.init(Cipher.DECRYPT_MODE, secretKey, ivSpec);

    String decryptedReportJson = new
String(aesCipher.doFinal(Base64.getDecoder().decode(secureDataBase64)),
StandardCharsets.UTF_8);

    java.security.PublicKey doorsPublicKey = resolveDoorsPublicKey(serverKeyIdHint !=
null ? serverKeyIdHint : "doors-pki-v1");

    java.security.Signature publicSignature =
java.security.Signature.getInstance("SHA256withRSA");
    publicSignature.initVerify(doorsPublicKey);
    publicSignature.update(decryptedReportJson.getBytes(StandardCharsets.UTF_8));

```

```

        if (!publicSignature.verify(Base64.getDecoder().decode(signatureBase64))) {
            throw new java.security.SignatureException(" 🚨 CRYPTOGRAPHIC FAULT:
Signature verification mismatch.");
        }

// 🟡 MODULE 2: DIGITAL SIGNATURE AUDIT TRAIL PANEL
System.out.println("\n-----");
System.out.println(" 🟡 DOORS PKI EMBEDDED DIGITAL SIGNATURE VERIFIED");
System.out.println(" 📦 Key Identifier Hint (KID) : " + (serverKeyIdHint != null ?
serverKeyIdHint : "UNKNOWN"));
System.out.println(" 🔑 Verification Engine State : SHA256withRSA (Algorithm
Validated)");
System.out.println(" 📄 Trunk Payload Signature : " +
truncateSignatureString(signatureBase64));
System.out.println("-----\n");

return decryptedReportJson;
}

private static String truncateSignatureString(String sig) {
    if (sig == null || sig.length() <= 32) return sig;
    return sig.substring(0, 16) + "... " + sig.substring(sig.length() - 16);
}

private static String readStream(java.io.InputStream is) throws Exception {
    if (is == null) return "";
    StringBuilder sb = new StringBuilder();
    try (BufferedReader br = new BufferedReader(new InputStreamReader(is,
StandardCharsets.UTF_8))) {
        String line; while ((line = br.readLine()) != null) sb.append(line.trim());
    }
    return sb.toString();
}

private static String extractJsonValue(String jsonStr, String propertyKey) {
    if (!jsonStr.contains(propertyKey)) return null;
    try {
        int startIndex = jsonStr.indexOf("\"" + propertyKey + "\":") + propertyKey.length() +
3;
        if (jsonStr.charAt(startIndex) == '"') {
            startIndex++;
            return jsonStr.substring(startIndex, jsonStr.indexOf('"', startIndex));
        } else {
            int endIndex = jsonStr.indexOf(',', startIndex);
            if (endIndex == -1) endIndex = jsonStr.indexOf('}', startIndex);
            return jsonStr.substring(startIndex, endIndex).trim();
        }
    }

```

```

    }
    } catch (Exception e) { return null; }
}

public static void main(String[] args) {
    try {
        String configUrl = "http://localhost:8052/api/v1/master/gateway";

        // 🛡️ RESOLVED DYNAMICALLY: Code contains no security text blocks
        String configKey = resolveSecurityKey();

        List<String> assignedAgents = java.util.Arrays.asList("Dev-01");

        System.out.println("===== RUNNING GENERATED DOORS DEVELOPMENT
CLIENT =====");
        GePNICDOORSAPIClientDevProfileGePNICInternal client = new
GePNICDOORSAPIClientDevProfileGePNICInternal(configUrl, configKey);
        client.executeHandshake();

        // Execution context for template: ORGWISE_USERS
        System.out.println("\n 🧩 Orchestrating dataset query: ORGWISE_USERS...");
        java.util.Map<String, String> map_ORGWISE_USERS = new
java.util.HashMap<>();
        StringBuilder jsonBuilder_ORGWISE_USERS = new StringBuilder("{}");
        int count_ORGWISE_USERS = 0;
        for (java.util.Map.Entry<String, String> entry : map_ORGWISE_USERS.entrySet()) {

            jsonBuilder_ORGWISE_USERS.append("\").append(entry.getKey()).append("\":\").app
end(entry.getValue()).append("\");
            if (++count_ORGWISE_USERS < map_ORGWISE_USERS.size()) {
                jsonBuilder_ORGWISE_USERS.append(",");
            }
        }
        jsonBuilder_ORGWISE_USERS.append("{}");
        String params_ORGWISE_USERS = jsonBuilder_ORGWISE_USERS.toString();

        String output_ORGWISE_USERS = client.fetchReportData("ORGWISE_USERS",
assignedAgents, params_ORGWISE_USERS);
        System.out.println(" 🧩 Response Resolved Successfully -> \n" +
output_ORGWISE_USERS);

        // Execution context for template: Bidders List
        System.out.println("\n 🧩 Orchestrating dataset query: Bidders List...");
        java.util.Map<String, String> map_Bidders_List = new java.util.HashMap<>();
        StringBuilder jsonBuilder_Bidders_List = new StringBuilder("{}");
        int count_Bidders_List = 0;

```

```

        for (java.util.Map.Entry<String, String> entry : map_Bidders_List.entrySet()) {

jsonBuilder_Bidders_List.append("\").append(entry.getKey()).append("\":\").append(e
ntry.getValue()).append("\");
        if (++count_Bidders_List < map_Bidders_List.size()) {
            jsonBuilder_Bidders_List.append(",");
        }
    }
    jsonBuilder_Bidders_List.append("}");
    String params_Bidders_List = jsonBuilder_Bidders_List.toString();

    String output_Bidders_List = client.fetchReportData("Bidders List",
assignedAgents, params_Bidders_List);
    System.out.println(" 🎉 Response Resolved Successfully -> \n" +
output_Bidders_List);

    System.out.println("\n===== 🏰 ALL CONTEXTUAL DATA ORCHESTRATIONS
PASSED PERFECTLY =====");
    } catch (Exception ex) {
        ex.printStackTrace();
    }
}

/**
 * 🏠 Resolves the active Gateway API Key passport safely.
 * Hierarchy: 1. System Environment Variable -> 2. ~/.security/doors.key
 */
private static String resolveSecurityKey() {
    String envKey = System.getenv("DOORS_API_KEY");
    if (envKey != null && !envKey.isBlank()) {
        return envKey.trim();
    }

    String userHome = System.getProperty("user.home");
    java.nio.file.Path secretFilePath = java.nio.file.Paths.get(userHome, "security",
"doors.key");
    java.io.File keyFile = secretFilePath.toFile();

    if (keyFile.exists() && keyFile.isFile()) {
        try {
            String loadedKey = java.nio.file.Files.readString(secretFilePath,
java.nio.charset.StandardCharsets.UTF_8);
            if (loadedKey != null && !loadedKey.isBlank()) {
                return loadedKey.trim();
            }
        } catch (Exception e) {

```

```

        System.err.println(" ❌ Secure IO Exception occurred while accessing
credentials: " + e.getMessage());
    }
}

    throw new SecurityException("DOORS BOOT FAULT: Active API Key Passport
missing. " +
        "Please configure DOORS_API_KEY env variable or provision " +
secretFilePath.toString() + " with 600 bounds.");
    }
}

```

4.2.5.7.2 Production Version of API Client

```

/**
 *
=====
=====
 * 🇧🇩 NATIONAL INFORMATICS CENTRE (NIC) — GePNIC - Data Orchestration API
Client
 * 📄 STANDARD OPERATING PROCEDURE (SOP) FOR CRYPTOGRAPHIC ASSET
MANAGEMENT
 * Disclaimer: This generated client template is provided as a reference implementation
 * to demonstrate secure integration practices with the DOORS API. It is the
 * responsibility of the implementer to ensure that all cryptographic materials
 * are managed in accordance with organizational security policies and industry
 * best practices.
 *
=====
=====
 * * 1. IDENTITY & CONFIGURATION SUMMARY
 * - Client Class: GePNICDOORSAPIClientProdProfileadmin
 * - Profile Mode: PRODUCTION PROFILE (Hardened KeyStore)
 * - Security Model: Digital Certificate Handshake with Outbound Data Payload
Encryption
 * *
 * * 2. HARDENED PRODUCTION SECURITY MANDATE & CRYPT-WALLET
PROVISIONING
 * What is a PKCS12 (.p12) File?
 * A .p12 (also known as a PFX container) is an industry-standard secure binary wallet
file.
 * Unlike raw plain-text .pem keys, a .p12 container wraps your Private Key and Public
X.509
 * Certificate together into a single encrypted bundle locked behind a strong password
token.
 * This ensures that even if an adversary gains raw disk access, your key materials
remain unreadable.

```

```

*
* How to construct a 'doors-identity.p12' using OpenSSL:
* If your organization possesses an out-of-band Private Key (private.key) and an X.509
* Public Certificate (certificate.crt / cert.pem), you can compile them into the exact
* PKCS12 format required by this JVM profile client by executing this local terminal
sequence:
*
* [Command Line Terminal Script Execution Pass]:
* $ openssl pkcs12 -export \
* -out doors-identity.p12 \
* -inkey private.key \
* -in certificate.crt \
* -name "doors-client-alias"
*
* Step A: Execution will prompt you for an Export Password. Choose a strong
passphrase token.
* Step B: Update the matching 'passToken' parameter variable string inside your main
runtime initialization block below.
*
* Deployment Target Configuration Path:
* Create an isolated system folder configuration boundary and move the keys safely:
* [Linux/Unix Terminal]:
* $ mkdir -p ~/security/
* $ mv doors-identity.p12 ~/security/
* $ echo "API_KEY_SHARED_BY_API_SERVICE_PROVIDER" > ~/security/doors.key
* $ chmod 600 ~/security/doors.key ~/security/doors-identity.p12
* * [Windows PowerShell]:
* > md "$HOME\security"
* > move doors-identity.p12 "$HOME\security\"
* > "API_KEY_SHARED_BY_API_SERVICE_PROVIDER" | Out-File -FilePath
"$HOME\security\doors.key" -Encoding ascii
*
* * * 3. PRODUCTION STAGING & DEPLOYMENT RUN INSTRUCTIONS
* Step 1 (Compile the structural package tree):
* $ javac -d . GePNICDOORSAPIClientProdProfileadmin.java
* * Step 2 (Execute the secure production binary class target):
* $ java org.gepnic.doors.masterapi.GePNICDOORSAPIClientProdProfileadmin
* * * 4. KEY ROTATION & APPLICATION RUNTIME OVERRIDES
* - Default Target Wallet Location: System.getProperty("user.home") + "/security/doors-
identity.p12"
* - To inject keystore configuration parameters natively from host environment
variables, modify your application instantiation:
* String activeWalletPath = System.getenv("DOORS_WALLET_PATH");
* String activeWalletPass = System.getenv("DOORS_WALLET_PASS");
* String activeWalletAlias = System.getenv("DOORS_WALLET_ALIAS");

```

```

* *
=====
=====
*/
package org.gepnic.doors.masterapi;

import javax.crypto.Cipher;
import javax.crypto.spec.IvParameterSpec;
import javax.crypto.spec.SecretKeySpec;
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.OutputStream;
import java.net.HttpURLConnection;
import java.net.URL;
import java.net.URLEncoder;
import java.math.BigInteger;
import java.nio.charset.StandardCharsets;
import java.security.KeyFactory;
import java.security.KeyPair;
import java.security.PublicKey;
import java.security.spec.RSAPublicKeySpec;
import java.util.Base64;
import java.util.List;

public class GePNICDOORSAPIClientProdProfileadmin {

    private final String baseUrl;
    private final String apiKey;
    private KeyPair liveKeyPair;
    private String publicKeyBase64;

    private PublicKey cachedDoorsPublicKey = null;
    private String cachedDoorsKeyId = null;

    public GePNICDOORSAPIClientProdProfileadmin(String baseUrl, String apiKey)
throws Exception {
        this.baseUrl = baseUrl.endsWith("/") ? baseUrl.substring(0, baseUrl.length() - 1) :
baseUrl;
        this.apiKey = apiKey;
        // HARDENED SECURITY MANDATE: Permanent keys are loaded explicitly during
application boot sequence context
    }

    // 🗝️ PRODUCTION KEYSTORE LOADER: Securely binds encrypted local wallets to
JVM execution memory
    public void loadPermanentIdentity(String keyStorePath, String password, String alias)
throws Exception {

```

```

        java.security.KeyStore ks = java.security.KeyStore.getInstance("PKCS12");
        try (java.io.FileInputStream fis = new java.io.FileInputStream(keyStorePath)) {
            ks.load(fis, password.toCharArray());
        }
        java.security.PrivateKey privateKey = (java.security.PrivateKey) ks.getKey(alias,
password.toCharArray());
        java.security.PublicKey publicKey = ks.getCertificate(alias).getPublicKey();
        this.liveKeyPair = new java.security.KeyPair(publicKey, privateKey);
        this.publicKeyBase64 =
Base64.getEncoder().encodeToString(publicKey.getEncoded());
    }

    public void executeHandshake() throws Exception {
        HttpURLConnection conn = (HttpURLConnection) new URL(this.baseUrl +
"/handshake").openConnection();
        conn.setRequestMethod("POST");
        conn.setRequestProperty("Content-Type", "application/json");
        conn.setRequestProperty("X-API-KEY", this.apiKey);
        conn.setDoOutput(true);

        String payload = String.format("{\"publicKey\": \"%s\"}", this.publicKeyBase64);
        try (OutputStream os = conn.getOutputStream()) {
            os.write(payload.getBytes(StandardCharsets.UTF_8));
        }

        if (conn.getResponseCode() != 200) {
            throw new RuntimeException("DOORS-HANDSHAKE-REJECTED.");
        }
    }

    private synchronized PublicKey resolveDoorsPublicKey(String requiredKeyId) throws
Exception {
        if (this.cachedDoorsPublicKey != null &&
requiredKeyId.equals(this.cachedDoorsKeyId)) {
            return this.cachedDoorsPublicKey;
        }

        HttpURLConnection conn = (HttpURLConnection) new URL(this.baseUrl + "/well-
known/jwks.json").openConnection();
        conn.setRequestMethod("GET");
        conn.setRequestProperty("X-API-KEY", this.apiKey);
        String jwksResponse = readStream(conn.getInputStream());

        String base64UrlModulus = extractJsonValue(jwksResponse, "n");
        byte[] modulusBytes = Base64.getUrlDecoder().decode(base64UrlModulus);
        byte[] exponentBytes =
Base64.getUrlDecoder().decode(extractJsonValue(jwksResponse, "e"));

```

```

        BigInteger modulus = new BigInteger(1, modulusBytes);
        BigInteger exponent = new BigInteger(1, exponentBytes);

        RSAPublicKeySpec rsaSpec = new RSAPublicKeySpec(modulus, exponent);
        this.cachedDoorsPublicKey =
KeyFactory.getInstance("RSA").generatePublic(rsaSpec);
        this.cachedDoorsKeyId = requiredKeyId;

        return this.cachedDoorsPublicKey;
    }

    public String fetchReportData(String queryUniqueName, List<String> targetAgentIds,
String criteriaParams) throws Exception {
        String encodedQuery = URLEncoder.encode(queryUniqueName,
StandardCharsets.UTF_8.toString()).replaceAll("\\\\+", "%20");
        HttpURLConnection conn = (HttpURLConnection) new URL(this.baseUrl +
"/orchestrate/" + encodedQuery).openConnection();
        conn.setRequestMethod("POST");
        conn.setRequestProperty("Content-Type", "application/json");
        conn.setRequestProperty("X-API-KEY", this.apiKey);
        conn.setDoOutput(true);

        StringBuilder agentsJson = new StringBuilder();
        agentsJson.append("[");
        for (int i = 0; i < targetAgentIds.size(); i++) {
            agentsJson.append("\"").append(targetAgentIds.get(i).trim()).append("\"");
            if (i < targetAgentIds.size() - 1) {
                agentsJson.append(",");
            }
        }
        agentsJson.append("]");

        String cleanParams = (criteriaParams == null || criteriaParams.trim().isEmpty()) ? "{}"
: criteriaParams.trim();
        String jsonRequestPayload = "{\"agentIds\": " + agentsJson.toString() + ", \"params\": "
+ cleanParams + "}";

        try (OutputStream os = conn.getOutputStream()) {
os.write(jsonRequestPayload.getBytes(StandardCharsets.UTF_8)); }

        int status = conn.getResponseCode();
        if (status == 403) {
            executeHandshake();
            return fetchReportData(queryUniqueName, targetAgentIds, criteriaParams);
        }

        if (status >= 400) {

```

```

        throw new RuntimeException("DOORS Error -> " +
readStream(conn.getErrorStream()));
    }

    String serverKeyIdHint = conn.getHeaderField("X-DOORS-KID");
    String response = readStream(conn.getInputStream());

    String secureDataBase64 = extractJsonValue(response, "secureData");
    String encryptedKeyBase64 = extractJsonValue(response, "encryptedKey");
    String ivBase64 = extractJsonValue(response, "iv");
    String signatureBase64 = extractJsonValue(response, "signature");

    if (secureDataBase64 == null) return response;

    // 🚪 MODULE 1: INBOUND WIRE DATA VERIFICATION PANEL

    System.out.println("\n=====
=====");
    System.out.println("🚪 INBOUND WIRE DATA VERIFICATION (RAW ENCRYPTED
BLOCKS)");

    System.out.println("=====
=====");
    System.out.println("🔒 Encrypted Session AES Key : " +
truncateSignatureString(encryptedKeyBase64));
    System.out.println("🔑 Initialization Vector (IV): " + ivBase64);
    System.out.println("📦 Ciphertext Payload Body : " +
truncateSignatureString(secureDataBase64));

    System.out.println("=====
=====");

    Cipher rsaCipher = Cipher.getInstance("RSA/ECB/PKCS1Padding");
    rsaCipher.init(Cipher.DECRYPT_MODE, this.liveKeyPair.getPrivate());
    byte[] rawAesKeyBytes =
rsaCipher.doFinal(Base64.getDecoder().decode(encryptedKeyBase64));

    Cipher aesCipher = Cipher.getInstance("AES/CBC/PKCS5Padding");
    SecretKeySpec secretKey = new SecretKeySpec(rawAesKeyBytes, "AES");
    IvParameterSpec ivSpec = new
IvParameterSpec(Base64.getDecoder().decode(ivBase64));
    aesCipher.init(Cipher.DECRYPT_MODE, secretKey, ivSpec);

    String decryptedReportJson = new
String(aesCipher.doFinal(Base64.getDecoder().decode(secureDataBase64)),
StandardCharsets.UTF_8);

```

```

    java.security.PublicKey doorsPublicKey = resolveDoorsPublicKey(serverKeyIdHint !=
null ? serverKeyIdHint : "doors-pki-v1");

    java.security.Signature publicSignature =
java.security.Signature.getInstance("SHA256withRSA");
    publicSignature.initVerify(doorsPublicKey);
    publicSignature.update(decryptedReportJson.getBytes(StandardCharsets.UTF_8));

    if (!publicSignature.verify(Base64.getDecoder().decode(signatureBase64))) {
        throw new java.security.SignatureException(" 🚨 CRYPTOGRAPHIC FAULT:
Signature verification mismatch.");
    }

    // 🌊 MODULE 2: DIGITAL SIGNATURE AUDIT TRAIL PANEL
    System.out.println("\n-----");
    System.out.println(" 🌊 DOORS PKI EMBEDDED DIGITAL SIGNATURE VERIFIED");
    System.out.println(" 📦 Key Identifier Hint (KID) : " + (serverKeyIdHint != null ?
serverKeyIdHint : "UNKNOWN"));
    System.out.println(" 🔑 Verification Engine State : SHA256withRSA (Algorithm
Validated)");
    System.out.println(" 📦 Trunk Payload Signature : " +
truncateSignatureString(signatureBase64));
    System.out.println("-----\n");

    return decryptedReportJson;
}

private static String truncateSignatureString(String sig) {
    if (sig == null || sig.length() <= 32) return sig;
    return sig.substring(0, 16) + "..." + sig.substring(sig.length() - 16);
}

private static String readStream(java.io.InputStream is) throws Exception {
    if (is == null) return "";
    StringBuilder sb = new StringBuilder();
    try (BufferedReader br = new BufferedReader(new InputStreamReader(is,
StandardCharsets.UTF_8))) {
        String line; while ((line = br.readLine()) != null) sb.append(line.trim());
    }
    return sb.toString();
}

private static String extractJsonValue(String jsonStr, String propertyKey) {
    if (!jsonStr.contains(propertyKey)) return null;
    try {

```

```

        int startIndex = jsonStr.indexOf("\"" + propertyKey + "\":") + propertyKey.length() +
3;
        if (jsonStr.charAt(startIndex) == '"') {
            startIndex++;
            return jsonStr.substring(startIndex, jsonStr.indexOf('"', startIndex));
        } else {
            int endIndex = jsonStr.indexOf(',', startIndex);
            if (endIndex == -1) endIndex = jsonStr.indexOf('}', startIndex);
            return jsonStr.substring(startIndex, endIndex).trim();
        }
    } catch (Exception e) { return null; }
}

public static void main(String[] args) {
    try {
        String configUrl = "http://localhost:8052/api/v1/master/gateway";

        // 🗝️ RESOLVED DYNAMICALLY: Production mode also reads the key safely from
local environments
        String configKey = resolveSecurityKey();

        List<String> assignedAgents = java.util.Arrays.asList("GePNIC-Assam");

        System.out.println("===== RUNNING GENERATED DOORS PRODUCTION
CLIENT =====");
        GePNICDOORSAPIClientProdProfileadmin client = new
GePNICDOORSAPIClientProdProfileadmin(configUrl, configKey);

        String walletPath = System.getProperty("user.home") + "/security/doors-
identity.p12";
        String passToken = "CHANGE_ME_TO_SECURE_PASSPHRASE";
        String keyAlias = "doors-client-alias";

        client.loadPermanentIdentity(walletPath, passToken, keyAlias);
        client.executeHandshake();

        System.out.println("\n 🏗️ Orchestrating fallback query...");
        String outputJson = client.fetchReportData("ORGWISE_USERS", assignedAgents,
("{}");
        System.out.println(" 🌈 Response Resolved Successfully -> \n" + outputJson);
        System.out.println("\n===== 🏗️ ALL CONTEXTUAL DATA ORCHESTRATIONS
PASSED PERFECTLY =====");
    } catch (Exception ex) {
        ex.printStackTrace();
    }
}
}

```

```

private static String resolveSecurityKey() {
    String envKey = System.getenv("DOORS_API_KEY");
    if (envKey != null && !envKey.isBlank()) return envKey.trim();

    String userHome = System.getProperty("user.home");
    java.nio.file.Path secretFilePath = java.nio.file.Paths.get(userHome, "security",
"doors.key");
    try {
        return java.nio.file.Files.readString(secretFilePath,
java.nio.charset.StandardCharsets.UTF_8).trim();
    } catch (Exception e) {
        throw new SecurityException("DOORS PRODUCTION FAULT: API Key Passport
missing at " + secretFilePath.toString());
    }
}
}

```

4.2.6 User to Agent Mapping

 Data Manager MODE

 datamanager@gmail.com

 **User-Agent Authorization Matrix**

Email / Username	Role	Authorized Agents	Action
test@nic.in	Developer	No Agents Mapped	Manage Access
testcaptcha@gmail.com	DataViewer	No Agents Mapped	Manage Access
captcha@gmail.com	DataViewer	No Agents Mapped	Manage Access
testcaptch@gmail.com	DataViewer	No Agents Mapped	Manage Access
testcapt@gmail.com	DataViewer	No Agents Mapped	Manage Access
testcaptca@gmail.com	Developer	No Agents Mapped	Manage Access
kganapathi@nic.in	DataManager	Dev-01	Manage Access


Powered by DOORS
A product from GePNIC


NIC
National Informatics Centre


GePNIC
eProcurement Platform

© 2026 National Informatics Centre
v1.0.3

Select agents from the left and move to the right to grant access.

☐ Available Agents 0/3

☐ GePNIC Staging 2 (GePNIC-Staging-2)

☐ UAT (GePNIC-UAT)

☐ Development Environment - 01 (Dev-01)

<

>

☐ Authorized 0/2

☐ Dev-02 (Dev-02)

☐ GePNIC-Assam (GePNIC-Assam)

Cancel

Update Permissions

4.2.7 DOORS – PKI Key Management

Two options are available – Can use internal generated key pair or Upload PKI code signing certificates.

Menu Data Manager MODE datamanager@gmail.com

PKI Signing Keys & Certificate Governance + Upload CA Certificate + Generate New Key Pair

Index	KEY PAIR DESCRIPTION NAME	KEY IDENTIFIER (KID)	DISK FILE STORAGE STATUS	GENERATED ON	ACTIVE SIGNER STATUS
1	Initial Core Bootstrap Keys	doors-pki-v1	doors-pki-v1.der doors-pki-v1.pem	5/23/2026, 3:50:46 PM	LIVE
4	Doors Demo Certificate	doors-demo-cert	doors-demo-cert.der doors-demo-cert.pem	5/23/2026, 6:38:46 PM	STAGED
3	NewKey-3	doors-pki-v3	doors-pki-v3.der doors-pki-v3.pem	5/23/2026, 5:42:21 PM	STAGED
2	NewKey-2	doors-pki-v2	doors-pki-v2.der doors-pki-v2.pem	5/23/2026, 5:32:58 PM	STAGED

4.2.7.1 Generate DOORS Internal PKI Certificate Pair

Automated Server PKI Generation Wizard

Secure Execution Boundary Note
Key Generation takes place natively inside the secure JVM server memory bubble. Private key vectors are written straight to disk storage and are never sent across the internet.

Unique Key Identifier String (KID-Version)
e.g., doors-pki-v2
Only alphanumeric characters, hyphens, and underscores are allowed.

Certificate Display Name Description
e.g., NIC Production Signing Token May 2026

Cancel Execute Generation Engine

doors-pki-v2

4.2.7.2 Upload DOORS PKI Code Signing Certificates

Import CA Certificate Bundle

CA Asset Verification Rule

!

Please upload the unencrypted PKCS#8 private key (.key/.pem) and the corresponding public X.509 certificate chain (.crt/.pem) issued by your Certifying Authority.

Unique Key Identifier String (KID-Version)

e.g., doors-nic-ca-v1

Provide a unique name to structure the asset names on disk.

Certificate Display Name Description

e.g., Certified Official Production Root

Private Key File (.key / .pem)

Public Certificate (.pem / .crt)

Browse...

No file selected.

Browse...

No file selected.

Cancel

Upload and Initialize

4.2.8 New Data Request by External User

New Data Orchestration Request

DOORS Protocol v2028

* Request Title / Subject (Brief Summary)

e.g., GePNIC Tender Data for Chennai Smart City - FY 2025-260 / 150

* Target Campus / Agent Node

Select destination node

* Business Justification & Data Scope

Provide a detailed business purpose for this data request.

Technical Context

Required Output Structure (Sample JSON)

e.g. { "sender_id": "string", "amount": 0.00 }

Authorization Evidence

Attach Signed Request or Approval Email

Drop evidence here or [click to upload](#)

Max 10MB per file. PDF/Email snapshots are required for compliance audit.

Clear Form

Submit to Governance Hub

4.2.9 Submit SQL Templates

- Paste /Type the SQL text
- Test the SQL by 'Dry-Run' on the assigned Sand Box Agent.
- If Found OK, store it by assigning unique name, Category and assigning at least one target agent.

Submit SQL Template for Governance

* Unique Name (System Reference)

* Category

Select Domain

Used as a slug for API endpoints.

* SQL Text (PostgreSQL Syntax) ⓘ

```
SELECT * FROM table WHERE city = :city
```

ⓘ Only SELECT statements are permitted for orchestration.

Development Environment - 01 (Sand...)

ⓘ Run Dry Run

* Description & Justification

Why is this data pull required?

0 / 500

Cancel

Submit for Review

4.2.10 Interactive Reports (Data Reporting) – Interactive Viewer

The screenshot shows the DOORS Data Manager interface. On the left is a dark sidebar with navigation links: My Dashboard, New Data Request, My Data Requests, SQL Submission, Governance Hub, Data Reporting, and Interactive Viewer. The main header includes the DOORS logo and navigation tabs: GOVERNANCE, INTEGRATION, and DATA SHARING. Below the header, a red bar indicates 'Data Manager MODE' and the user 'datamanager@gmail.com' is logged in. The main content area is titled 'Interactive Report Viewer' and features a search bar, a dropdown for 'Select Report' (currently 'ORGWISE_USERS'), a dropdown for 'Target Agent' (currently 'Development Environment - 01 (Dev-01)'), and a 'Generate' button. Below these are 'Export Excel' and 'Export CSV' buttons. A table displays 1331 records, with the first 8 rows visible. The table has columns: NODE, NAME, ORGANISATION, DESIGNATION, MOBILENO, and ALTERNATEMAILID. The footer includes logos for 'Powered by DOORS', 'NIC', and 'GePNIC', along with copyright information for the National Informatics Centre and version 'v1.8.3'.

NODE	NAME	ORGANISATION	DESIGNATION	MOBILENO	ALTERNATEMAILID
Dev-01	Abhishek Sharma	NIC	Manager	7240835808	abhi15190@gmail.com
Dev-01	Abhishek Sharma	NIC	Manager	7240835808	abhi15190@gmail.com
Dev-01	Rajesh S	NIC	AEE	9447344926	rajeshkwa2010@gm...
Dev-01	pravesh bhardwaj	NIC	PM	9999999999	dgdept4@gmail.com
Dev-01	Itishree Pattanaik	NIC	JE	9776933753	pattanaik.itishree9@g...
Dev-01	eproc ten	NIC	td	9000000000	eproc.rajasthan@gm...
Dev-01	Raj Demo2	NIC	Test	7073507367	eproc.rajasthan@gm...
Dev-01	AJITH KUMAR SINGH	NIC	EE	9433230135	dhhwdn@yahoo.com

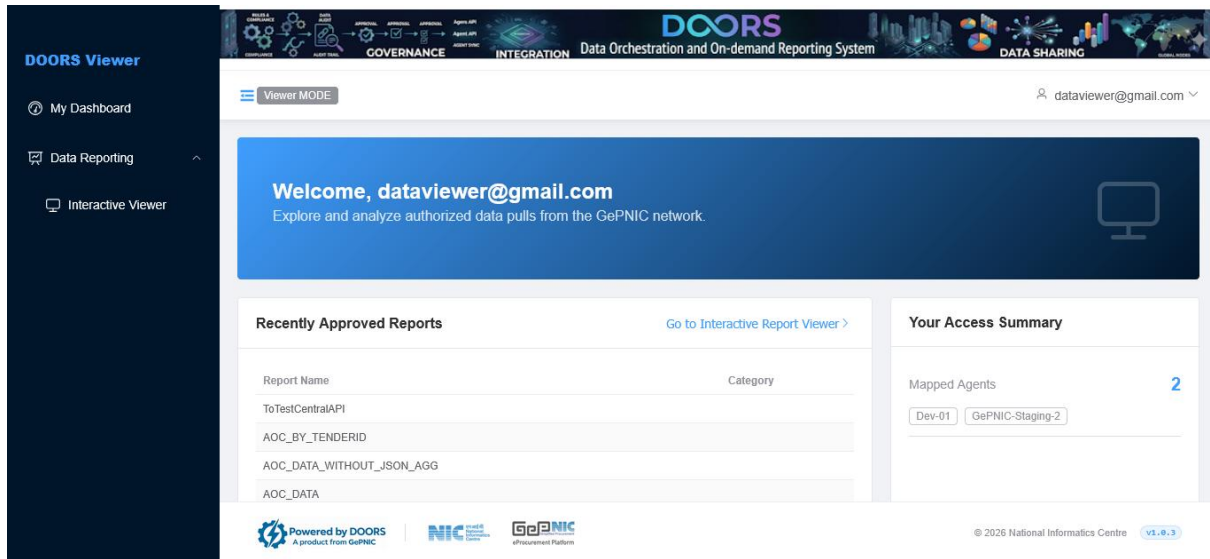
4.2.11 SQL Template Category Master

The screenshot shows the DOORS Data Manager interface for the 'Query Category Master Data' section. The left sidebar is the same as in the previous screenshot. The main header is identical. The main content area is titled 'Query Category Master Data' and includes an 'Add New Category' button. Below this is a table with columns: ID, CATEGORY NAME, SYSTEM CODE, and ACTIONS. The table lists several categories with their corresponding system codes and action icons (edit and delete). The footer is the same as in the previous screenshot.

ID	CATEGORY NAME	SYSTEM CODE	ACTIONS
3	Infrastructure Health	INFRA	[Edit] [Delete]
4	Monthly Status Reports	MSR	[Edit] [Delete]
6	On Demand Report	ON_DEMAND	[Edit] [Delete]
1	MIS-Reports	MIS	[Edit] [Delete]
2	Security Audit Logs	AUDIT	[Edit] [Delete]
5	Executive Dashboard	DASHBOARD	[Edit] [Delete]

4.3 Data Viewer Menus

(i) Dashboard



The screenshot shows the DOORS Viewer Dashboard. The left sidebar contains the following menu items: **DOORS Viewer**, **My Dashboard**, **Data Reporting**, and **Interactive Viewer**. The main content area has a header with "Viewer MODE" and a user profile "dataviewer@gmail.com". Below the header is a welcome message: "Welcome, dataviewer@gmail.com. Explore and analyze authorized data pulls from the GePNIC network." The dashboard is divided into two main sections: "Recently Approved Reports" and "Your Access Summary".

Recently Approved Reports

Report Name	Category
ToTestCentralAPI	
AOC_BY_TENDERID	
AOC_DATA_WITHOUT_JSON_AGG	
AOC_DATA	

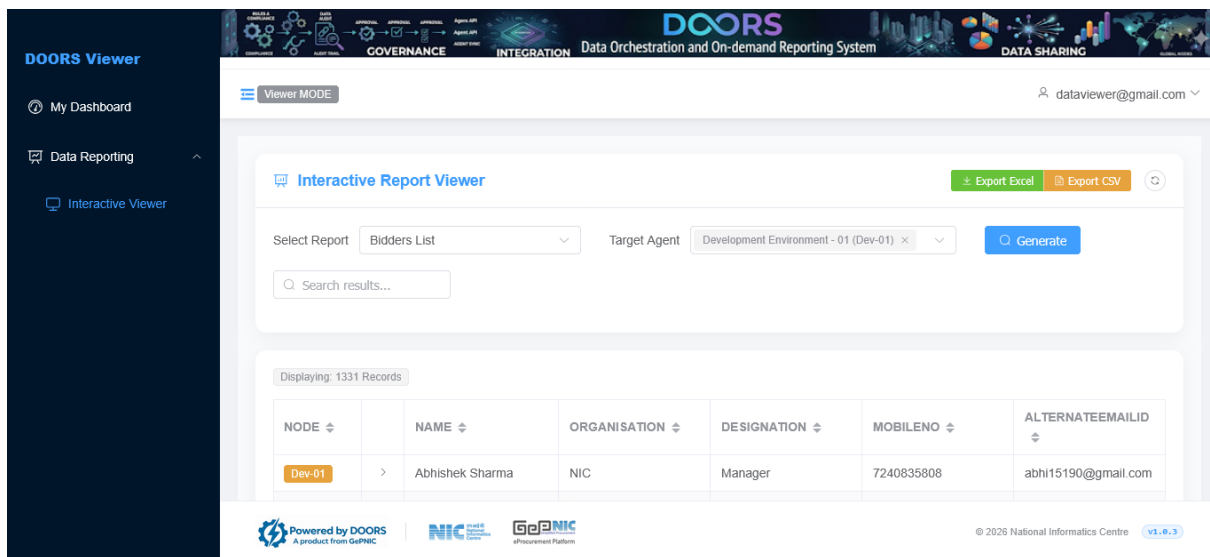
Your Access Summary

Mapped Agents: 2

Dev-01 | GePNIC-Staging-2

Footer: Powered by DOORS A product from GePNIC, NIC, GePNIC eProcurement Platform, © 2026 National Informatics Centre, v1.6.3

(ii) Report Viewer



The screenshot shows the DOORS Viewer Report Viewer. The left sidebar is the same as the dashboard. The main content area has a header with "Viewer MODE" and a user profile "dataviewer@gmail.com". Below the header is the "Interactive Report Viewer" section. It includes a "Select Report" dropdown menu with "Bidders List" selected, a "Target Agent" dropdown menu with "Development Environment - 01 (Dev-01)" selected, and a "Generate" button. There are also "Export Excel" and "Export CSV" buttons. Below the filters is a search bar labeled "Search results...".

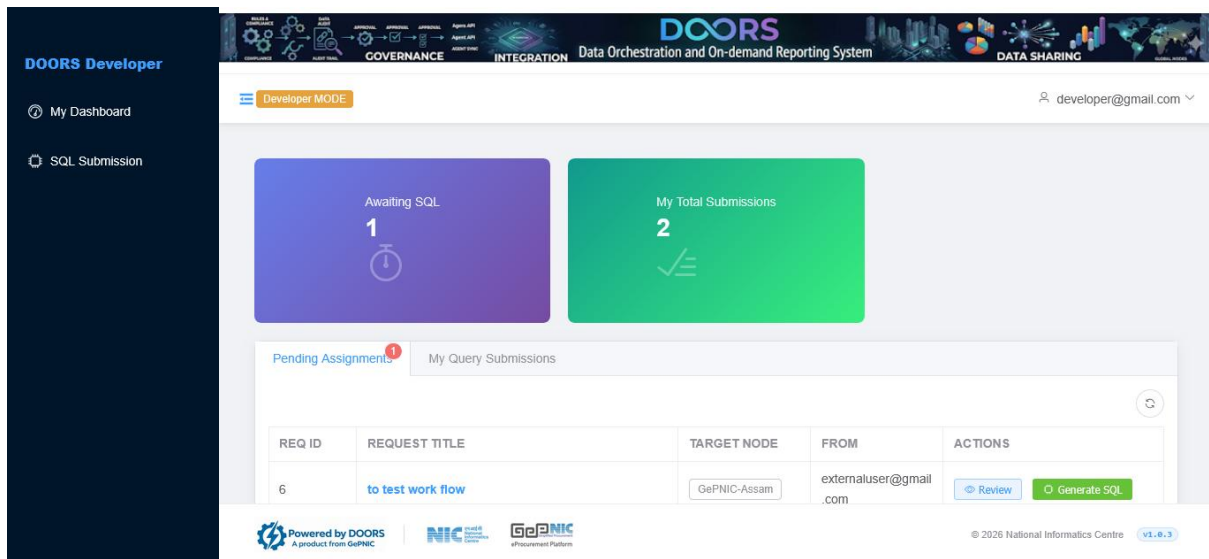
Displaying: 1331 Records

NODE	NAME	ORGANISATION	DESIGNATION	MOBILENO	ALTERNATEEMAILID	
Dev-01	>	Abhishek Sharma	NIC	Manager	7240835808	abhi15190@gmail.com

Footer: Powered by DOORS A product from GePNIC, NIC, GePNIC eProcurement Platform, © 2026 National Informatics Centre, v1.6.3

4.4 Developer Menu

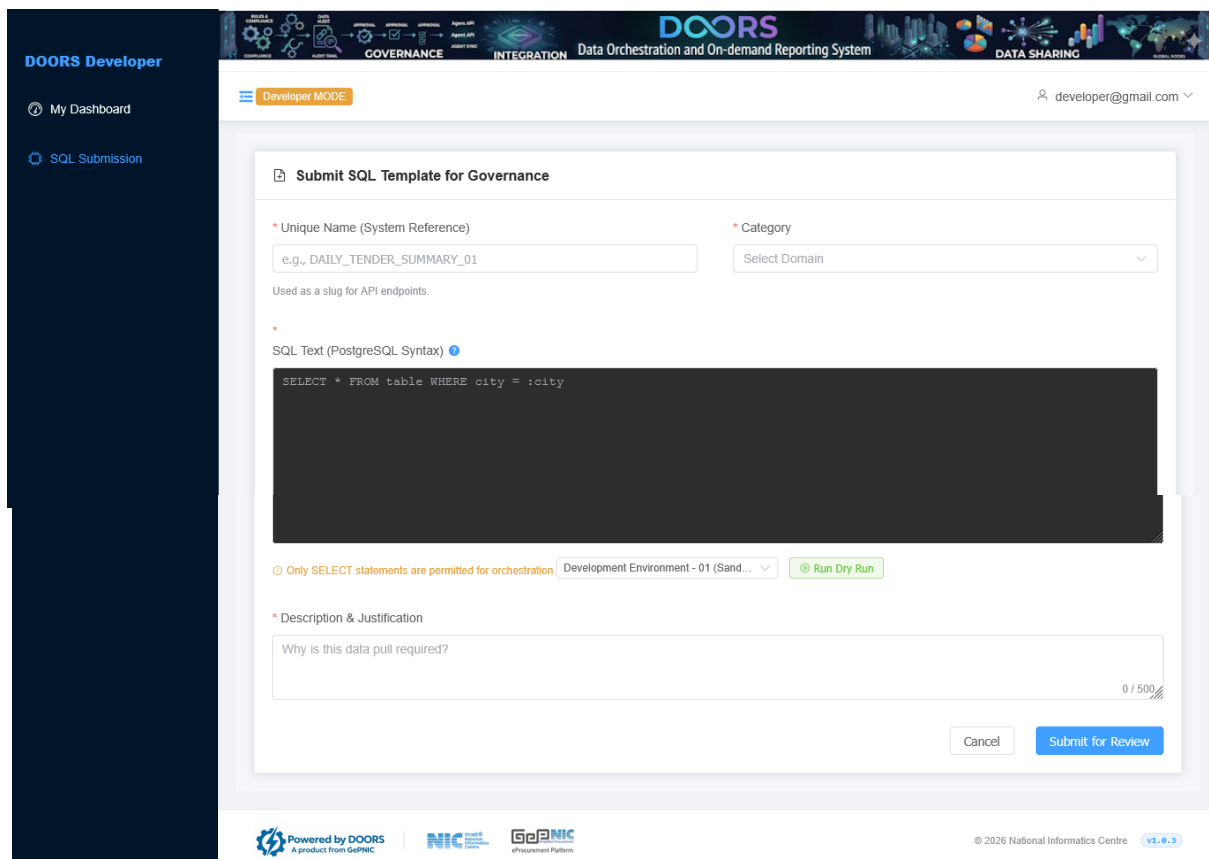
(i) Dashboard



The screenshot shows the DOORS Developer Dashboard. The left sidebar contains the 'DOORS Developer' logo and two menu items: 'My Dashboard' and 'SQL Submission'. The main content area is titled 'Developer MODE' and shows the user 'developer@gmail.com'. It features two large cards: 'Awaiting SQL' with a count of 1 and a clock icon, and 'My Total Submissions' with a count of 2 and a checkmark icon. Below these is a 'Pending Assignments' section with a red notification badge, showing a table of query submissions. The table has columns for REQ ID, REQUEST TITLE, TARGET NODE, FROM, and ACTIONS. One submission is listed with REQ ID 6 and REQUEST TITLE 'to test work flow'. The ACTIONS column contains 'Review' and 'Generate SQL' buttons. The footer includes logos for 'Powered by DOORS', 'NIC', and 'GePNIC', along with copyright information for the National Informatics Centre and version 'v1.0.3'.

REQ ID	REQUEST TITLE	TARGET NODE	FROM	ACTIONS
6	to test work flow	GePNIC-Assam	externaluser@gmail.com	Review Generate SQL

(ii) SQL Submission



The screenshot shows the 'Submit SQL Template for Governance' form in the DOORS Developer application. The form is titled 'Submit SQL Template for Governance' and includes the following fields and controls:

- Unique Name (System Reference):** A text input field containing 'e.g., DAILY_TENDER_SUMMARY_01'.
- Category:** A dropdown menu with 'Select Domain' as the selected option.
- SQL Text (PostgreSQL Syntax):** A text area containing the SQL query: `SELECT * FROM table WHERE city = :city`.
- Environment:** A dropdown menu showing 'Development Environment - 01 (Sand...)'.
- Run Dry Run:** A green button labeled 'Run Dry Run'.
- Description & Justification:** A text area with the placeholder text 'Why is this data pull required?' and a character count of '0 / 500'.
- Buttons:** 'Cancel' and 'Submit for Review' buttons.

The footer of the page includes logos for 'Powered by DOORS', 'NIC', and 'GePNIC', along with copyright information for the National Informatics Centre and version 'v1.0.3'.

4.5 External User Menu

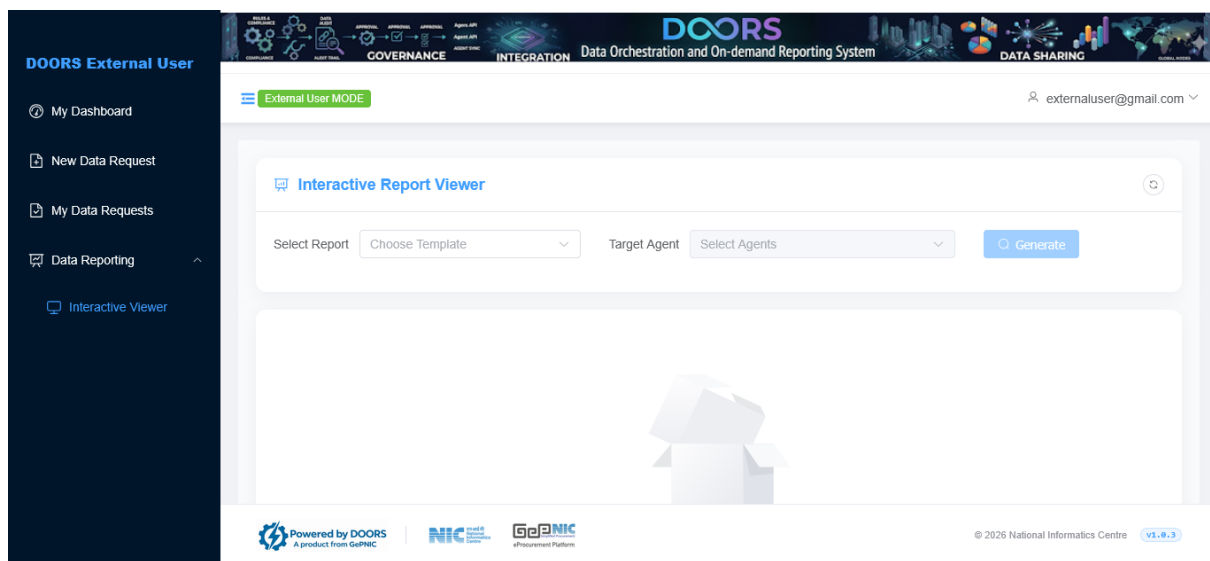
(i) Dashboard

The screenshot shows the DOORS External User Dashboard. The header includes the DOORS logo and navigation links for GOVERNANCE, INTEGRATION, and DATA SHARING. The left sidebar lists: My Dashboard, New Data Request, My Data Requests, and Data Reporting. The main content area features a welcome message for 'externaluser@gmail.com' with buttons for 'New Data Request' and 'View My Submissions'. Below this is a 'Request Process' section with a 4-step flow: 1. Submit Request (Define data parameters), 2. Governance Review (DataManager validation), 3. SQL Orchestration (Developer query prep), and 4. Data Availability (Access via API). A 'Quick Support' section provides a link to 'Contact GePNIC Data Manager' and a note about security protocols. The footer contains logos for DOORS, NIC, and GePNIC, along with copyright information for the National Informatics Centre.

(ii) New Data Request Form

The screenshot displays the 'New Data Orchestration Request' form. The form is titled 'New Data Orchestration Request' and includes a 'DOORS Process Link' button. It contains several sections: 'Request Title / Subject (Brief Summary)' with a text area and character count; 'Target Campus / Agent Node' with a dropdown menu; 'Business Justification & Data Scope' with a text area; 'Technical Content' with a 'Required Output Structure (Sample JSON)' field; and 'Authorization Evidence' with a 'Attach Signed Request or Approval Email' field and a file upload button. At the bottom, there are 'Clear Form' and 'Submit to Governance Hub' buttons. The footer includes logos for DOORS, NIC, and GePNIC, and copyright information for the National Informatics Centre.

(iii) Report Viewer



5. DOORS – Repository

- DOORS-UI ->
- DOORS-MasterAPI ->
- DOORS-AgentAPI->

6. Production Deployment

It is suggested that DOORS UI & backend services may be run as containers and DB in a VM ;

- New DOORS production domain **doors.gepnic.nic.in** may be registered
- VUE.js UI should be built as container and run on Open Shift Container Platform (OCSP)
- Master API should also be built as spring boot container with Open JDK 21 and run on OCSP - /home/doors/doors-master-api/config/application.yml @ 10.246.129.226 should be used
- Agent API should be installed in any one of the VMs under each VLAN ;
 - o Install openjdk 21.0.10 2026-01-20 LTS from RHEL Repo

```
#dnf install java-openjdk
```

- o Create a user 'doors' & login as doors
- o Create a deployment directory /home/doors/doors-agent-api
- o Deploy doorsagentapi.jar under /home/doors/doors/doors-agent-api
- o - /home/doors/doors-agent-api/config/application.yml @ 10.246.129.226 should be used
- o Create a Service file
/etc/systemd/system/ doorsagentapi.service

With the content;

```
[Unit]
```

```
Description=DOORS-Agent-API Spring Boot Service
```

```
After=network.target
```

```
[Service]
```

```
User=root
```

```
WorkingDirectory=/home/doors/doors-agent-api
```

```
ExecStart=java -jar /home/doors/doors-agent-api/doorsagentapi.jar >> /var/log/doorsagentapi.log 2>&1
```

```
SuccessExitStatus=143
```

```
Restart=on-failure
```

```
StandardOutput=journal
```

```
StandardError=journal
```

```
[Install]
```

```
WantedBy=multi-user.target
```

7. Major Configurations:

7.1 Session Time Out @ UI:

The session timeout is hard coded on the UI code @ MainLayout.vue in this code->

```
function resetTimer() {  
  clearTimeout(timeout);  
  // 15 minutes of inactivity  
  timeout = setTimeout(() => {  
    ElMessageBox.alert('You have been logged out due to inactivity.',  
'Session Expired');  
    performLogoutRedirect(); // Your helper function  
  }, 15 * 60 * 1000);  
}
```

7.2 Master API Configurations

- LogFile, Connection Timeout , Origin Site, Server Port, DB Connection, rate-limit, jwt-expiration, gateway root path, Security key directory.

All these configuration are set in application.yml file;

----- application.yml file @ production -----

```
port: 8052  
forward-headers-strategy: native  
max-http-header-size: 64KB # Handle large metadata/column counts  
tomcat:  
  connection-timeout: 600000 # 10 minutes (matches your Apache ProxyTimeout)  
  keep-alive-timeout: 600000  
crypto:  
  keys-directory: ./security/keys/  
  
spring:  
  datasource:  
    url: jdbc:postgresql://10.246.129.229:5455/doors_registry  
    username: doorsuser  
    password:  
    driver-class-name: org.postgresql.Driver  
  servlet:  
    multipart:  
      max-file-size: 100MB # Increased from 10MB  
      max-request-size: 100MB # Increased from 10MB
```

```
jpa:
  hibernate:
    ddl-auto: update # Automatically creates tables from your Java Entities
  show-sql: true
  properties:
    hibernate:
      format_sql: true

threads:
  virtual:
    enabled: true
```

Portal Registry

```
doors:
  security:
    allowed-origins:
      - "http://localhost:*"
      - "http://127.0.0.1:*"
      - "http://demoetenders.tn.nic.in:*"
      - "https://demoetenders.tn.nic.in"
  rate-limit:
    # Policy for Human Interactive Users (UI)
  ui:
    capacity: 10      # Number of clicks allowed in the burst
    refill-minutes: 1 # How often the bucket refills fully
    penalty-minutes: 1 # (Logic-only) Duration for strict lockout if needed
  # Policy for the Trusted Orchestration Server (API Gateway)
  gateway:
    capacity: 50      # Much higher for automated system loops
    refill-minutes: 1
  jwt-expiration-ms: 14400000 #4 hours
  # Define which paths are considered "Gateway"
  # This keeps your Filter code clean
  gateway-path-pattern: "/v1/master/gateway/orchestrate/"
app:
  portals:
    - id: "PORTAL_01"
      name: "Main Portal"
      url: http://10.246.129.229:8051
  security:
    enabled: false # Set to false to allow browser testing from any IP
    # This is your whitelist. Add your Master API's IP here.
    # You can override this in production using the environment variable:
    ALLOWED_MASTERS
```

```

    allowed-masters:
${ALLOWED_MASTERS:10.246.129.229,10.246.129.225,localhost,127.0.0.1,0:0:0:0:0:0:0:1}
logging:
# 📁 1. PERSISTENCE & AUTO-ROTATION INFRASTRUCTURE
file:
    # Bound explicitly to your NIC production folder directory asset path
    name: /var/log/doors/masterapi.log
logback:
    rollingpolicy:
        # Keep up to 30 days of text history for security compliance and audit trails
        max-history: 30
        # Automatically rotate and split the file when it reaches 10MB to keep file handling
nimble
    max-file-size: 10MB
    # Cap total historical storage footprint across all compressed files to 2GB max
    total-size-cap: 2GB
    # 🚀 Background GZ Compression: Stamping pattern forces automated .gz archival
compression on split
    file-name-pattern: /var/log/doors/archived/masterapi-%d{yyyy-MM-dd}.%i.log.gz}

# 🏠 2. HIGH-FIDELITY ENGINE TRACKING BOUNDARIES
level:
    root: INFO
    org.gepnic.doors: INFO

# 🌐 A. Netty Wire Data Pipeline (Tracks raw inbound/outbound traffic bytes)
reactor.netty.http.client: DEBUG
reactor.netty.http.server: DEBUG

# ⚡ B. Spring Reactive Function Memory Buffers (Monitors 256KB constraint
thresholds)
org.springframework.web.reactive.function.client.ExchangeFunctions: TRACE
org.springframework.http.codec: TRACE

# 🗄️ C. Relational Persistence Audits (Preserves your active database query
timelines)
org.springframework.jdbc.core.JdbcTemplate: DEBUG
org.springframework.jdbc.datasource.DataSourceUtils: DEBUG
org.hibernate.orm.results: DEBUG
org.hibernate.type.descriptor.sql.BasicBinder: TRACE

```

7.3 Agent API Configurations

- Server Port, Log File, Master API White Listing are configured at application.yml;

```
----- Agent application.yml -----

server:
  port: 8051 # Common to run Agents on 8081 to avoid conflict with Master (8080)

spring:
  threads:
    virtual:
      enabled: true # Enables Java 21 Virtual Threads for high-concurrency streaming

app:
  security:
    enabled: false # Set to false to allow browser testing from any IP
    # This is your whitelist. Add your Master API's IP here.
    # You can override this in production using the environment variable:
    ALLOWED_MASTERS
    allowed-masters:
      ${ALLOWED_MASTERS:10.246.129.229,localhost,127.0.0.1,0:0:0:0:0:0:1}
  logging:
    # 📁 1. PERSISTENCE & COMPACT ROTATION INFRASTRUCTURE
    file:
      # Bound to your dedicated local agent log path directory
      name: /var/log/doors/agentapi.log
    logback:
      rollingpolicy:
        # Retain 15 to 30 days of text history for local node compliance audits
        max-history: 15
        # Rotate and split files at 10MB to prevent heavy file reads on state servers
        max-file-size: 10MB
        # Cap total historical storage footprint on the node disk to 1GB max
        total-size-cap: 1GB
        # 🚀 Background GZ Compression: Automatically gzips older logs on rotation split
        file-name-pattern: /var/log/doors/archived/agentapi-%d{yyyy-MM-dd}.%i.log.gz
```

🏠 2. NODE-SPECIFIC ENGINE TRACKING BOUNDARIES

level:

root: INFO

org.gepnic.doors.agent: INFO

📁 A. Relational Data Pool Tracking (Monitors local query execution timelines)

org.springframework.jdbc.core.JdbcTemplate: DEBUG

org.springframework.jdbc.datasource.DataSourceUtils: DEBUG

🚰 B. Connection Pool Monitoring (Crucial for detecting database dropout/timeouts)

com.zaxxer.hikari: WARN

org.hibernate.engine.jdbc.connections.internal: WARN

🌐 C. Cross-DC Handshake & Network Boundaries

org.springframework.web.client.RestTemplate: INFO

org.apache.http.client5.http: WARN

7.4 Footer Customization

Footer Consists of five components. Out of these, four components can be customized by replacing the following images of the front document root ;

branding/logo-left.png

branding/logo-center.png

branding/logo-right.png

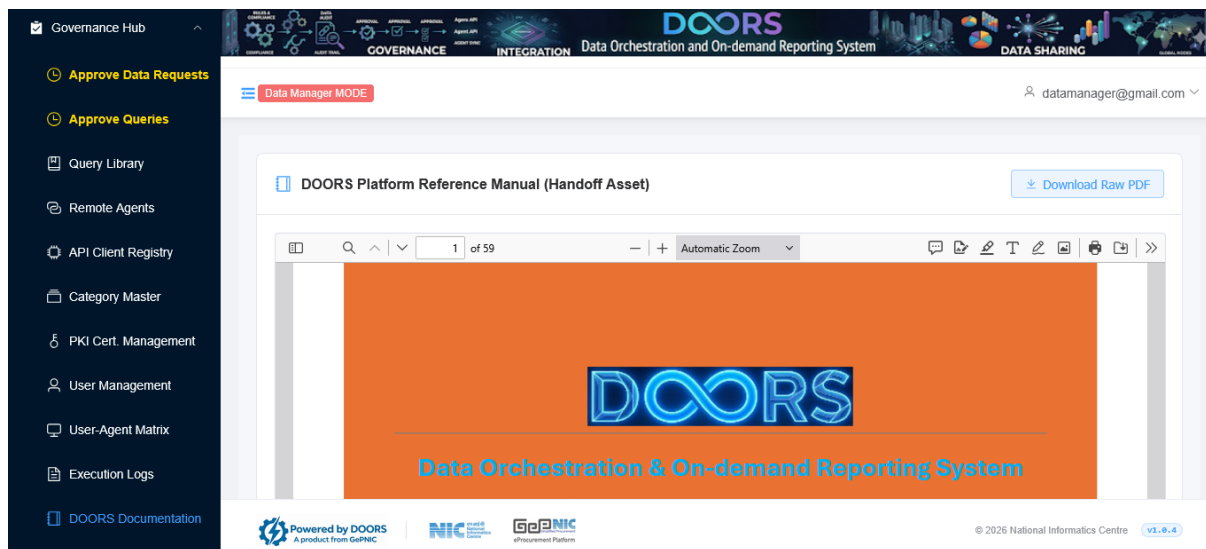
branding/logo-copyright.svg

The fifth component is the DOORS version number which comes from UI code as hardcoded.

----- Footer -----

8. Online DOORS Document

This document is published as DOORS.pdf under public/docs/ folder and made accessible from UI menu link 'DOORS Documentation' under 'Governance Hub'. This can be viewed or downloaded.



End of  Document